

DEPARTMENT OF MATHEMATICS

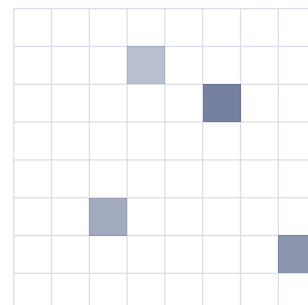
并行数值计算

课程笔记

Parallel Numerical Computing

作者： 隴千夏

日期： 2026 年 5 月 9 日



目录

1 并行计算机体系结构	1
1.1 并行计算机的分类	1
1.2 SIMD 型并行机	1
1.3 MIMD 型并行机	1
1.3.1 共享存储 MIMD 并行多处理机	1
1.3.2 分布存储 MIMD 并行多处理机	2
1.3.3 分布共享存储 MIMD 并行多处理机	3
2 并行算法	4
2.1 概述	4
2.2 数值并行算法引例	5
2.2.1 例 1. SIMD 算法	5
2.2.2 例 2. 同步 MIMD 算法	6
2.2.3 例 3. 异步 MIMD 算法	7
2.3 并行算法的复杂性	9
2.3.1 运行时间 $t(n)$	9
2.3.2 处理机台数 $P(n)$	9
2.4 算法的并行度	9
2.5 并行算法的加速比	11
2.6 并行算法的相容性和独立性	12
3 线性递推问题并行计算	15
3.1 倍增法	15
3.2 循环加倍法	17
4 并行矩阵-向量乘法	20
4.1 两种计算方式	20
4.2 向量计算分析	21
4.3 并行计算分析	21
4.4 实际选型讨论	21
5 矩阵乘积并行计算	23
5.1 内积、中积与外积算法	23
5.1.1 内积算法	23
5.1.2 中积算法	24
5.1.3 外积算法	24
5.1.4 三种算法的比较	25
5.2 Strassen 算法	26
5.3 Winograd 算法	28
6 并行矩阵求逆	30
6.1 一般稠密矩阵求逆的消去法	30

6.1.1	Marshall 格式	30
6.2	分块递推法	31
6.2.1	上三角矩阵的情形	32
7	线性方程组的并行求解	34
7.1	三角方程组的并行求解	34
7.1.1	行扫描法与列扫描法	34
7.1.2	列扫描法的并行实现	35
7.1.3	块列扫描法	36
7.2	一般线性方程组的并行求解	36
7.2.1	Gauss 消去法	37
7.2.2	列主元消去法	38
7.2.3	LU 分解的向量形式	38
7.2.4	LU 分解的加边算法	40
7.2.5	Dongarra-Eisenstat 算法	41
7.2.6	LU 分解矩阵与多重右端项	42
7.3	三对角线性方程组的并行解法	43
7.3.1	Stone 算法	44
7.3.2	循环约化法	44
7.3.3	分裂法	46
7.3.4	分段追赶并行算法	49
7.4	实对称方程组的 Cholesky 分解法	50
7.4.1	对称正定方程组的 Cholesky 分解法	50
7.4.2	ijk 型向量算法	51
7.4.3	Cholesky 分解的并行实现	51
7.4.4	QR 分解简介	52
7.4.5	Givens 约化法	53
7.4.6	Givens 约化法的向量与并行实现	54
7.4.7	Householder 约化法	55
7.4.8	Householder 约化法的向量与并行实现	57

1 并行计算机体系结构

1.1 并行计算机的分类

并行计算机主要指两台或两台以上的处理机连接起来可以并行操作的机器, 亦可称之为并行多处理机。

按照指令流和数据流的不同, 可以把并行计算机分成下面两类:

1. 单指令流多数据流 (SIMD): 各处理机同一时刻执行相同的指令, 处理不同的数据。
2. 多指令流多数据流 (MIMD): 各处理机同一时刻执行不同的指令, 且处理不同的数据。

CSAPP 中这样定义并发 (concurrency) 和并行: “We use the term concurrency to refer to the general concept of a system with multiple, simultaneous activities, and the term parallelism to refer to the use of concurrency to make a system run faster.” 并发是一个依赖抽象层次的概念, 我们有指令级并行, 线程级并发, 但在课程笔记中我们不会强调这一点。

1.2 SIMD 型并行机

SIMD 型并行机主要有以下三类:

1. 向量流水线处理机: 一般通过时间重叠技术实现并行处理; 把一个功能部件分成几个不同的部分, 每一部分执行部分功能。
2. 阵列处理机: 由成千上万个功能非常简单的处理机构成, 数据以某种方式流经各台处理机, 由它们进行处理。
3. 并行机: 通过一台前端机执行指令, 而并行机根据指令对数据进行处理, 如 CM-2、Mas Par MP-1、MP-2。

1.3 MIMD 型并行机

促使 MIMD 结构发展的因素主要有以下两点:

1. 支持更高的并行度 (子程序级、任务级或更高)。
2. 多台处理机的性价比远比一台处理机高。

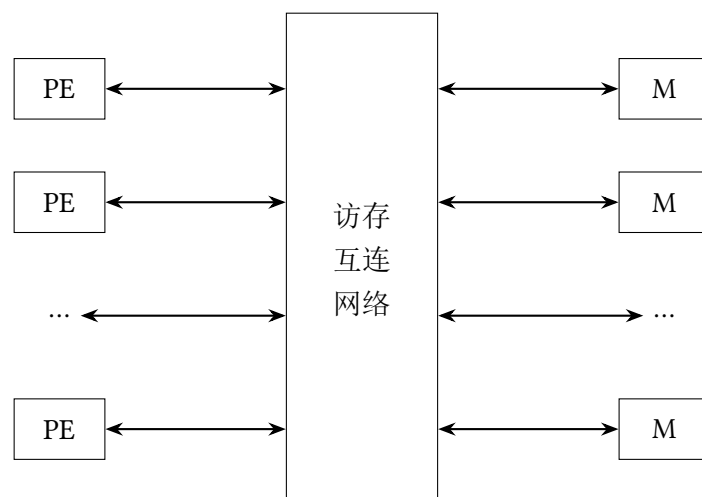
按照存储方式的不同, 我们可以将 MIMD 型并行机可以分成以下三类:

1. 共享存储并行多处理机。
2. 分布存储并行多处理机。
3. 分布共享存储并行多处理机。

1.3.1 共享存储 MIMD 并行多处理机

共享存储 MIMD 并行多处理机是指将多台处理机通过访存互连网络共享一个统一的内存空间, 并通过该内存空间来实现处理机间协调的并行计算系统, 也称为对称多处理机 (SMP, Symmetric Multiple Processors)。

共享存储 MIMD 并行多处理机的内存空间也可由多个存储器模块构成, 其结构如图所示:



在该类并行机中, 每台处理机可以执行相同或不同的指令流, 可以直接访问到所有数据, 处理机间通信是借助共享主存来实现的。

链接处理机与存储模块之间的网络类型大致可分为以下三种:

1. **总线互连结构**: 分时共享总线为每台处理机提供了访问内存模块的均等机会, 一次只能有一台处理机访问总线; 为了增加处理机台数, 可以采用多总线结构。
2. **Crossbar 互连结构**: 使用交叉开关, 在每个可能的处理机/存储器模块之间提供专用通道, 从而防止出现访存竞争冲突。这种互连配置价格较贵, 处理机数目一般在 4-16 台之间。
3. **多级互连网络结构**: 它是 Crossbar 和总线性价比的折中。通过多级互连开关互连网络 (由多级小型交换单元级联构成, 数据通过各级单元按地址逐段选路转发) 连接处理机与存储器模块; 相对于前面两种结构, 具有更好的可扩展性, 但内存访问延迟时间较长。

在共享存储 MIMD 并行机中, 处理机和内存模块分别位于互连网络的两侧, 每台处理机通过互连网络访问内存模块, 且所有处理机的访问方式都是一样的, 机会均等。同时, 还可以在每台处理机中设置少量的局部 Cache, 以减轻互连网络的负载。我们称这种共享存储结构为一致内存访问 (UMA, *Uniform Memory Access*) 结构。

UMA 结构的优点在于给每台处理机提供同样的访存带宽、访存机会和访存延迟时间, 处理机与内存模块相对于互连网络呈对称分布, 处理机之间是完全等价的, 由此可以解释为什么我们也称之为 SMP。

共享存储 MIMD 并行多处理机的主要问题是其可扩展性差且不可避免地遇到内存访问瓶颈的问题, 当处理机需要同时访问共享全局变量时, 产生内存竞争现象, 严重影响效率。其优点是容易进行并行程序设计, 能保持较好的负载平衡, 适合许多中小规模应用问题的计算和事务处理。

1.3.2 分布存储 MIMD 并行多处理机

分布存储 MIMD 并行多处理机中的每台处理机都有自己可直接访问的内存 (局部存储器)。每台处理机称为系统中的一个节点, 节点之间通过互连网络相互连接。每台处理机只能直接访问局部存储器, 对远端存储器的访问必须以消息传递的方式, 通过互连网络来完成。互连网络通常有总线互连结构、环结构、树结构。

注: 共享存储 MIMD 并行多处理机的 (访存) 互连网络和分布存储 MIMD 并行多处理机的互连网络在概念上是有所区别的, 在讲到互连网络时注意区分。

分布式存储并行机具有很好的可扩展性,是实现超大规模科学与工程计算的唯一途径. 如果要求获得较高性能,必须通过消息传递来进行并行程序设计,并行程序设计复杂,不容易被用户接受.

消息传递型分布式存储 MIDI 并行机主要有下面两类:

1. **大规模并行计算机 (MPP):** 由成百上千的功能相同的处理机通过互连网络连接而成, 如: IBM SP-2, Intel Paragon XP/S, Cray T3D, 国产 YH-3 等.
2. **网路并行机群系统:** 由同构或异构型串行或并行计算机通过快速局域网或广域网相互松散连接而成, 如: 工作站机群、PC 机群等.

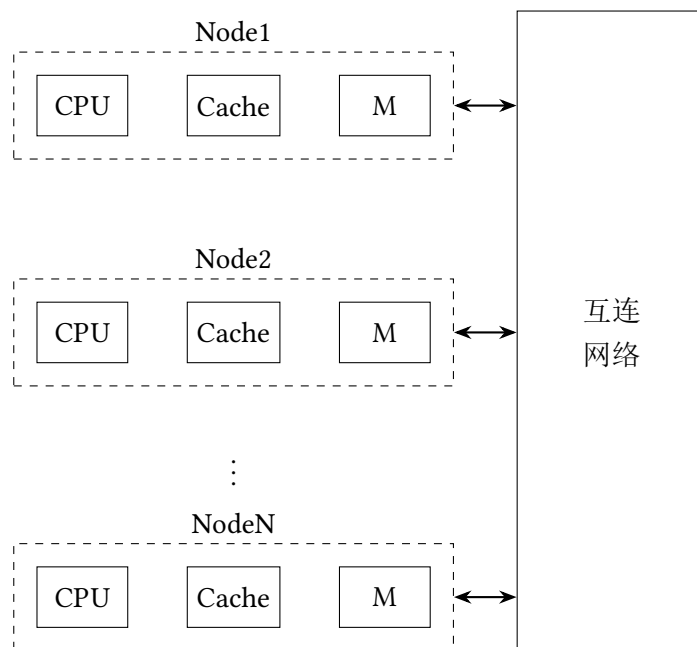
由于工作站机群、PC 机群充分利用各个单位自己的现有资源来组织并行计算, 特别适合于财力不足的科研单位和大学研究所, 近年来得到了快速的发展.

1.3.3 分布共享存储 MIMD 并行多处理机

为了结合 MPP 的可扩展性和 SMP 的并行程序设计的容易性, 提出一种新的结构, 即分布共享存储 (DSM) MIMD 并行计算机.

DSM 结构将 SMP 的内存模块分布到每台处理机, 成为它们的局部内存, 构成一个同一的节点, 节点之间通过互连网络相互连接. 每台处理机可以直接访问它的局部内存, 也可以通过互连网络直接访问别的处理机的局部内存, 只是延迟时间较长.

DSM 提供给用户的是统一的内存空间, 但在物理上类似于 MPP 结构, 是分布内存的, 故称之为分布共享存储结构并行机, 也称之为非一致内存访问 (NUMA, *Non-Uniform Memory Access*) 结构.



DSM 结构具有较好的可扩展性, 可以减轻 SMP 结构的访存瓶颈, 但是又继承了其并行程序设计的容易性.

DSM 的关键技术在于如何保持处理机内部的 Cache 及内存之间的一致性, 不解决这些问题同样会阻碍 DSM 的可扩展性和性能的发挥.

2 并行算法

2.1 概述

算法是解题方法的精确描述,它是一组有穷的规则,这些规则规定了解决某一特定类型问题的一系列运算.所谓并行算法,可以简单地认为是在各类并行计算机上求解问题的算法.

为了比较精确地给出并行算法的定义,我们需要用到计算机科学中的进程 (Process) 的概念.一个进程是指一段 (串行) 程序连同其数据在 (单) 处理机上的动态执行过程.简单地说,进程可以理解成顺序执行的一组操作或一段程序.

我们还会遇到和进程密切相关的一个概念: 线程 (Thread). 在 *Programming Massively Parallel Processors* 一书中这样定义线程: “The sequence of instruction execution activities resulting from this sequential, stepwise execution of an application is referred to as a thread of execution, or simply thread.” 可以看到,线程是执行的最小单位,线程不独立拥有内存地址空间,它共享进程的资源.进程是一种抽象,它被定义为 “one or more threads and their execution state”. 一个进程可以分为多个线程,进程是资源分配的最小单位.有时候 “进程” 一词用于指代只有一个线程的进程.

接下来给出并行算法的精确定义: 并行算法是一些可同时执行的诸进程的集合,这些进程相互作用和协调动作,从而达到对给定问题的求解.

并行算法可以从不同的角度加以分类:

1. 数值和非数值并行算法.
2. 同步和异步并行算法.
3. SIMD 和 MIMD 并行算法等.

数值并行算法

数值计算是指基于代数关系的一类计算问题,基本上属于数值分析 (对以数字形式表达的问题求解) 的范畴,研究数值计算问题的并行算法称为数值并行算法.

非数值并行算法

非数值计算是指基于关系运算的一类计算问题,基本上属于符号 (如字符、数字图形或别的一些记号) 处理的范畴,研究非数值计算问题的并行算法称为非数值并行算法.

同步算法

同步算法 (Synchronized Algorithm) 是指某些进程必须等待别的进程的一类并行算法. 因为一个进程的执行依赖于输入数据和系统中断,所以全部进程均必须同步在一个给定的时钟,以等待最慢的进程. 一般而言,运行在 SIMD 计算机上的并行算法为同步并行算法 (Synchronous Parallel Algorithm).

异步算法

异步算法 (Asynchronized Algorithm) 是指进程执行一般不必相互等待的一类并行算法. 异步算法具有一下性质:

1. 有一个可以为所有进程存取的整体变量集合.
2. 当进程的一个阶段完成后,这个进程首先读某些整体变量,然后根据这些变量的值及上一阶段刚得到的结果,该进程修改某些整体变量,接着启动下阶段或结束它本身.

在异步算法中, 进程之间的通信是通过整体变量可共享数据来实现的. 进程之间没有同步算法那样明显的依赖关系. 异步算法的主要特征是它的进程永不等待输入, 而只根据整体变量的最新信息来继续.

SIMD 算法

面向 SIMD 计算机的并行算法就是 SIMD 算法. SIMD 算法依据“P 台处理机在同一时刻对各自的数据执行同一指令”, 具有明显的同步性, 属于同步并行算法.

MIMD 算法

面向 MIMD 计算机的并行算法就是 MIMD 算法. MIMD 算法依据“P 台处理机在同一时刻对各自的数据执行不同指令”. MIMD 算法有同步和异步之区别.

由于使用并行和向量计算机, 我们面临的一个挑战: 在设计算法时, 必须充分利用具体并行计算机的特征.

原有的一些最好的串行算法变成了不能令人满意的算法, 需要重新改造甚至抛弃. 而在串行机上许多远不是最优的“老”算法, 由于其并行性而焕发了“青春”.

传统的数值方法正在经历并行性的改造并在机器上得以检验, 发展新的并行算法的研究工作正在向纵深发展.

2.2 数值并行算法引例

由于本课程为并行数值计算, 我们主要关注于数值并行算法.

数值并行算法的研究内容主要包含:

1. 在传统数值计算方法基础上进行并行性改造和创新.
2. 已经具有明显并行性的传统数值方法和新方法在并行机和向量机上的算法实现和程序设计.

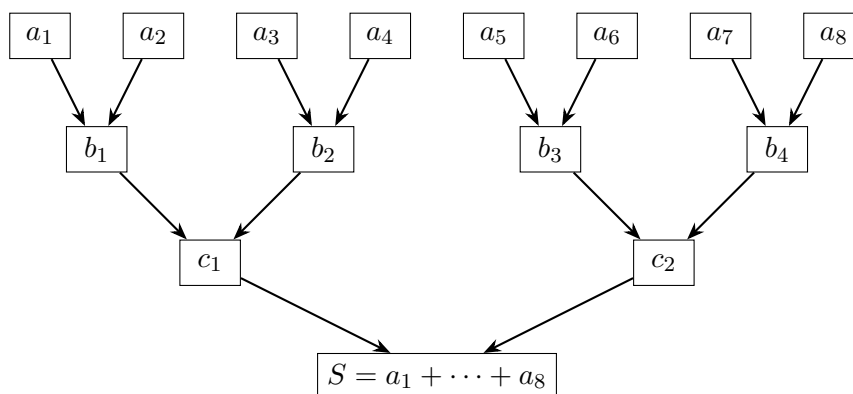
2.2.1 例 1. SIMD 算法

考虑 n 个数 $a_1, \dots, a_n$ 的求和问题. 其串行算法为:

```
s ← a1
for i = 2 to n do
    s ← s + ai
end for
```

显然不适合并行计算.

下图给出了 $n = 8$ 时的一种并行算法, 称为**结合扇入算法**, 其可由如下 4 个进程 P_1, P_2, P_3, P_4 组成



P_1	P_2	P_3	P_4
$b_1 \leftarrow a_1 + a_2$	$b_2 \leftarrow a_3 + a_4$	$b_3 \leftarrow a_5 + a_6$	$b_4 \leftarrow a_7 + a_8$
$c_1 \leftarrow b_1 + b_2$		$c_2 \leftarrow b_3 + b_4$	
$S \leftarrow c_1 + c_2$			

2.2.2 例 2. 同步 MIMD 算法

考虑求一元方程 $f(x) = 0$ 的根的 Newton 迭代法:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad i = 0, 1, 2, \dots$$

我们将每次迭代分成三个算法元:

$$y_i \leftarrow f(x_i)$$

$$y'_i \leftarrow f'(x_i)$$

$$x_{i+1} \leftarrow x_i - y_i / y'_i \text{ 及检验精度}$$

在一般情况下, 计算 $f(x_i)$ 和 $f'(x_i)$ 所含操作步数是不同的. 因此, 在具有两台速度相同的处理机系统上, 根据公式进行并行计算时, 每次迭代都须同步.

假设计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作, 下图给出了一种两进程同步 MIMD 算法:

时间步	P_1	P_2
1	$y_0 = f(x_0)$	$y'_0 = f'(x_0)$
2		
3	x_1 和检验	
4	$y_1 = f(x_1)$	$y'_1 = f'(x_1)$
5		
6	x_2 和检验	
$\vdots$	$\vdots$	$\vdots$

2.2.3 例 3. 异步 MIMD 算法

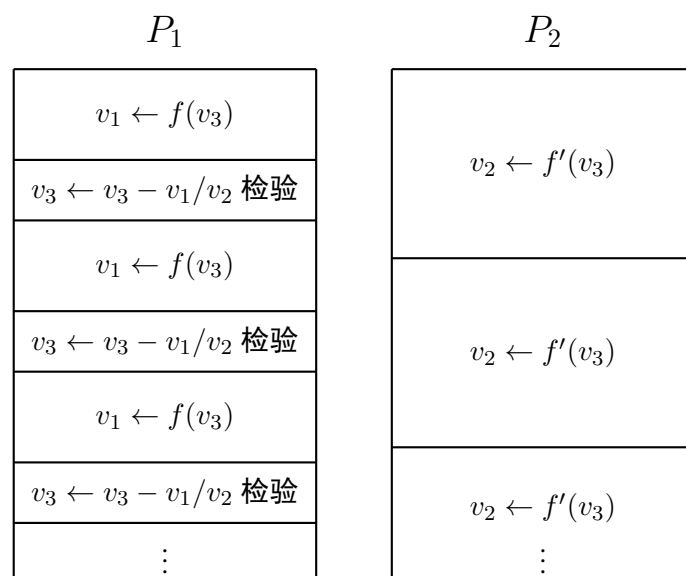
通过公用变量或公用数据来实现进程间的通信是异步算法的显著特征. 异步算法的进程间没有像同步算法那样有明显的依赖性, 各进程不需要时间以等待信息输入, 不论公用变量中当前信息是什么, 各进程将根据这些信息决定继续执行或终止.

为了保证逻辑上的正确性和进行有意义的工作, 如使必须更新的若干公用变量的更新过程全部完成或解决公用变量的存储冲突等, 必须允许异步算法中出现某些随机的临时封锁.

由于公用变量更新信息有很大的随机性, 直接法不能包含这种量, 所以异步算法基本上是迭代法, 并且与传统迭代法有很大区别.

仍以求 $f(x) = 0$ 的根为例, 引进公用变量 v_1, v_2, v_3 , 分别含 $f(x), f'(x), x$ 的当前值. 仍设计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作, 下图所示是一个比较合理的两进程异步 MIMD 算法, 或称为异步迭代算法:

P_1 更新 v_1 和 v_3 , 且检验精度, P_2 更新 v_2 .



假定变量的初值 $v_1 = 0, v_2 = f'(x_0), v_3 = x_1$, 则依上图计算时, v_3 得出的根的近似值序列如下:

$$\begin{aligned}
 x_2 &= x_1 - \frac{f(x_1)}{f'(x_0)} \\
 x_3 &= x_2 - \frac{f(x_2)}{f'(x_1)} \\
 x_4 &= x_3 - \frac{f(x_3)}{f'(x_2)}
 \end{aligned}$$

因此, 递推关系形如:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_j)}$$

其中 $j < i$, 当 $i \rightarrow \infty$ 时, $j \rightarrow \infty$, 但事先无法确定 j 和 i 的关系, 而且此递推形式只适合“计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作”的情形.

这已经不是传统的 Newton 迭代法, 而是一种异步迭代法, 或者可称之为混乱迭代法, 故必须对产生的序列 $\{x_i\}_{i=1}^{\infty}$ 的性质进行深入细致的分析.

按照传统的概念, 数值计算方法和程序设计是两门密切相关的不同学科.

算法是解题方法的精确描述, 它是一组有穷的规则, 这些规则确定了在计算机上求解某一问题的一系列运算. 根据算法可以直接地进行程序设计.

对于串行机而言, 数值方法 (可简称为解法) 与算法的关系比较简单, 解法一旦确定, 有效的串行算法和计算程序也随之产生其可在任何一种串行机上得到实现. 在并行计算系统出现以后, 区别解法和算法这两个概念的重要性增加了, 同样一个解法针对不同类型的并行机可以有着差异很大的算法和程序设计.

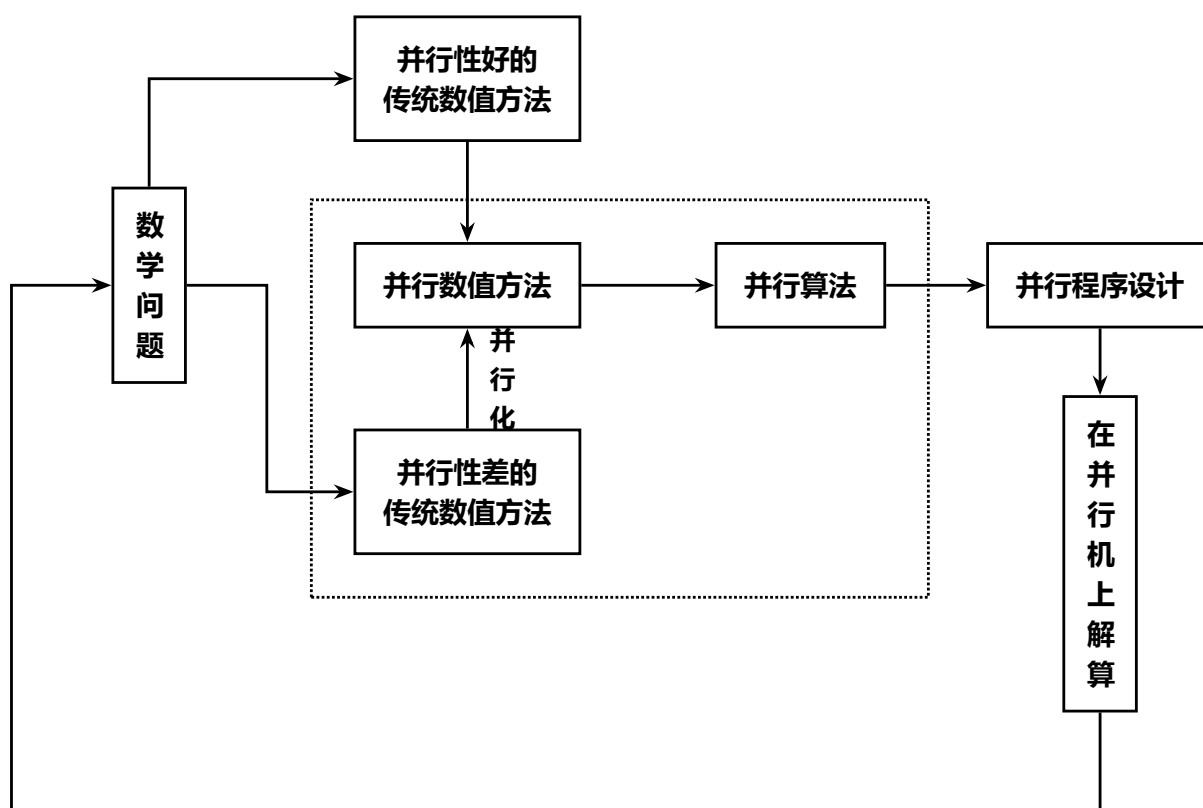
例如, 考虑求解 n 阶稠密线性代数方程组 $Ax = b$ 的 Gauss 消元法. 消元过程如下:

设 $a_{ii} \neq 0$, 要求通过矩阵初等变换将 a_{ji} ($i < j \leq n$) 化为零, 其余元素 a_{kl} 和右端元素 b_k ($i < k, l \leq n$) 都要进行修正, 变成 a_{kl}^* 和 b_k^*

对于串行情况, Gauss 解法的算法设计比较容易做到, 并且在实际应用上可以找到主元素 Gauss 消元法的标准子程序.

对于并行计算系统, Gauss 解法的算法实现可能有许多途径. 需要考虑到机器的类型和特点, 是并行机还是向量机, 是共享存储类型还是局部存储类型, 并行机的处理器个数有多少, 各处理器之间的负载平衡等问题.

并行算法一经给出, 并行程序即可直接由该算法设计产生. 算法受到数值方法影响和制约, 解法对算法有决定意义. 有效的并行算法一般必须建立在容易并行实现的数值方法基础上, 具有良好并行性的数值方法称为并行数值方法.



虚线所界定的内容构成了一个与并行算法直接相关的相当广泛的研究领域. 为了在并行机上实现给定问题的求解, 需要完成图中所示任务的一个大循环.

对于所要求解的数学问题, 如果传统数值方法已具备良好的和明显的并行性, 则可直接进入并行算法设计; 倘若传统方法没有良好的并行性, 则应对其进行并行性改造和创新.

具备并行性的传统数值方法

用主元素 Gauss 消元法解稠密线性代数方程组、矩阵与向量的乘法运算、求解发展方程的显式差分格式、求解线性代数方程组的 Jacobi 迭代法等

难于并行化的传统数值方法

递推问题、求解抛物型方程的隐式差分格式、椭圆型方程的直接近似解法和若干迭代解法等

这些难于并行化的传统数值方法在大规模科学计算中占有重要地位, 其并行化倍受重视, 近年来有了很大的发展, 提出了一些新技术和新方法. 同时也产生了许多新的理论问题, 如: 并行迭代法的收敛性、收敛速度的估计、并行化差分格式的相容性、收敛性、稳定性等. 这些问题都需要进一步深入的研究和分析.

需要指出, 并行数值方法的发展并不局限于上述所示框图, 例如不少学者在数学模型一级进行并行化研究, 利用分而治之的原则建立了求解数学物理问题的区域分裂法. 甚至为了并行计算的目的, 能够建立全新的计算模型, 如流体力学计算的格子气方法等.

传统数值方法在并行化方面正面临着新的挑战, 也面临着理论上取得突破和新发展的良好机遇. 并行计算新时期的出现孕育着并行数值方法大发展时期的到来, 并行数值方法的新领域目前正在呈现出蓬勃发展的势头.

2.3 并行算法的复杂性

一个算法的复杂性 (Complexity) 也叫复杂度, 是指其所含的工作量.

例如, 串行算法的复杂性是指算法的运算量 (即时间步) 与存储量 (存储空间), 其都是求解问题规模的函数. 当问题的规模 n 趋向无穷大时, 就得到算法复杂性的渐近表示. 一般我们关心的是 n 较大时的复杂性, 这时它与渐近复杂性相差不大.

对于在 SIMD 计算机和 MIMD 计算机上的并行算法, 我们研究算法的运行时间和所需处理机的台数与问题规模 n 的变化关系.

2.3.1 运行时间 $t(n)$

运行时间就是求解问题所需要的时间, 即算法从开始运行到结束运行所经过的时间. 如果不同处理机不能同时结束计算, 则并行算法的时间定义为从第一台处理机开始执行到最后一台处理机执行完所经过的时间.

通常在计算机上实现一种算法 (不管是串行算法还是并行算法) 之前, 需要在理论上分析实现这种算法所需的时间. 即我们需要统计算法所需执行的基本操作步数, 也即计算步和通信步, 分别用 t_c 和 t_r 表示.

2.3.2 处理机台数 $P(n)$

分析算法时通常假设求解给定问题所需的处理机数目是问题规模 n 的函数. 在有些场景, 有时处理机台数 P 是不依赖于 n 的常数, 并有时 P 远远小于 n .

2.4 算法的并行度

一个计算步的并行度是指该算法中不存在数据依赖关系、因而能用一个计算步完成的最大运算或操作个数.

注: 老师叫这个定义为算法的并行度, 由于对算法并行性的度量用平均并行度较为合理, 所以我将这个定义叫做计算步的并行度.

并行度的上限在并行机上受限于可同时运算的处理机数目, 在向量机上则受限于向量寄存器的最大长度.

例如, 计算两个 n 维向量 a 和 b 之和, 由于 n 个加法 $a_i + b_i$ 是独立的, 可以用一个加法步完成, 所以其并行度为 n .

算法一般需要用多个计算步, 当各步所含的运算个数均为 k 时, 算法的并行度等于 k ; 否则, 对一个算法并行性的度量只能按如下定义的平均并行度进行: 算法的平均并行度是指该算法总的操作数除以计算步数 (在向量机上体现为平均向量长度).

例如, 对于 $n = 2^q$ 个数的求和问题, 利用结合扇入加法需 $q = \log_2 n$ 个加法步, 并行性由高到低分布, 逐步减半, 其平均并行度为

$$\frac{1}{q} \left(\frac{n}{2} + \frac{n}{4} + \cdots + 1 \right) = \frac{n-1}{\log_2 n} = O\left(\frac{n}{\log_2 n}\right)$$

与并行度和向量化度有关的概念有粒度. 大粒度意味着能独立并行执行的是大任务, 而小粒度相反. 例如, n 阶线性方程组的迭代求解 (n 较大) 可作为大粒度的例子; 而两个 n 维向量的求和, 其每一个任务是两个标量之和, 可作为小粒度的例子.

向量处理有 3 种基本方式, 考虑下面这个例子:

```
for  $i = 1$  to  $N$  do
     $f_i \leftarrow a_i^2 b + d_i(a_i^2 - c_i)$ 
end for
```

(1) 横向加工

我们依次计算

$$\begin{aligned} a_1^2 * b + d_1 * (a_1^2 - c_1) &\rightarrow f_1 \\ a_2^2 * b + d_2 * (a_2^2 - c_2) &\rightarrow f_2 \\ &\dots \\ a_N^2 * b + d_N * (a_N^2 - c_N) &\rightarrow f_N \end{aligned}$$

一般计算机就是采用这种方式组成循环处理. 其共需 N 次循环, 不适合于作并行处理.

(2) 纵向加工

我们用黑体字母表示向量, 如 $\mathbf{a} = (a_1, \dots, a_n)$, 有些向量机 (如 STAR-100) 采用此方式:

$$\begin{aligned} \mathbf{a}^2 &\rightarrow \mathbf{u} \\ b\mathbf{u} &\rightarrow \mathbf{v} \\ \mathbf{u} - \mathbf{c} &\rightarrow \mathbf{u} \\ \mathbf{u} * \mathbf{d} &\rightarrow \mathbf{u} \\ \mathbf{u} + \mathbf{v} &\rightarrow \mathbf{f} \end{aligned}$$

(3) 纵横加工

令向量长度 $N = kn + r, n \leq N, r < n$. 对 $i = 1, 2, \dots, k$, 计算

$$\mathbf{a}_i^2 \rightarrow \mathbf{u}$$

$$b\mathbf{u} \rightarrow \mathbf{v}$$

$$\mathbf{u} - \mathbf{c}_i \rightarrow \mathbf{u}$$

$$\mathbf{u} * \mathbf{d}_i \rightarrow \mathbf{u}$$

$$\mathbf{u} + \mathbf{v} \rightarrow \mathbf{f}_i$$

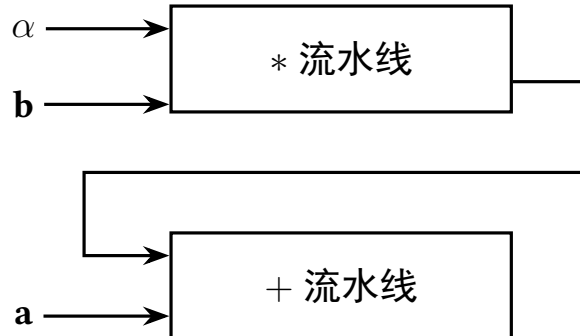
其中 $\mathbf{a}_i = (a_{in+1}, \dots, a_{in+n})$, $(0 \leq i < k)$, 而 $\mathbf{a}_k = (a_{kn+1}, \dots, a_N)$. $\mathbf{u}, \mathbf{v}$ 维数与 $\mathbf{a}_i$ 相同. 有些处理机 (如 CRAY-1、YH-1) 采用此方式.

纵向加工和纵横加工适合向量处理. 纵向加工向量长度不受限制, 中间结果需送回存储器, 对存储器的信息流量要求很高. 纵横加工对长向量分组处理, 由向量寄存器保留中间结果, 对存储器信息流量的要求可降低, 从而可提高处理速度.

对于向量计算机来说, 链接是一种十分吸引人的特征. 它允许将一条流水线的运算结果直接送至另一条流水线而无需先送回寄存器或存储器, 因此它可以非常有效地实现某些类型的向量运算, 例如链三元组等.

注: 链三元组是指形如 $\mathbf{a} + \alpha\mathbf{b}$, $(\mathbf{a} + \alpha)\mathbf{b}$ 之类的运算. 链接特性使这些运算几乎与向量加法或数乘具有相同的速度.

我们用下图表示利用链接来实现 $\mathbf{a} + \alpha\mathbf{b}$ 运算:



2.5 并行算法的加速比

加速比是并行算法研究的重要概念之一. 定义这个概念的目的是度量一个并行算法的并行性质. 并行算法的加速比定义为:

$$S_p = \frac{\text{串行算法在单处理机上执行的时间}}{\text{并行算法在具有 } P \text{ 台处理机的系统上的执行时间}}$$

为了准确估计算法的执行时间, 需要充分考虑到数据通信.

例如 n 个数相加, 使用结合扇入算法和 $n/2$ 台处理机, 每台处理机均有局部存储器. 假定 a_1, a_2 存放在 P_1 中, a_3, a_4 存放在 P_2 中...在进行第二步加法之前, 必须做数据传输, 也即是把 $a_3 + a_4$ 送到 P_1 中, $a_7 + a_8$ 送到 P_3 中. 如果假定一个加法执行的时间为 t , 数据传输时间为 αt ($\alpha > 1$), 则 n 个数相加的结合扇入算法加速比为:

$$S_p = \frac{(n-1)t}{(1+\alpha)t \log_2 n} = \frac{1}{1+\alpha} \frac{n-1}{\log_2 n}$$

上式表明, 在处理机台数 $P(n) = n/2$ 情况下, 结合扇入算法的加速比是并行度除以因子 $(1 + \alpha)$. 当 α 与 1 接近时, 通信与算数运算占同样多的时间, 则 S_p 是并行度的一半.

如果考虑向量计算机, 加速比定义中分子为算法的标量计算时间, 分母为向量计算时间. 另外, 在定义中, 分子有时也取为最快的穿行算法在单处理机上的执行时间, 虽然这样定义比较合理, 但是由于在许多情况下难以判断哪一个串行算法最快, 所以并不便于使用. 在定义中, 所研究的并行算法可以与任何给定的串行算法进行比较, 在实际使用时比较方便.

显然, S_p 越大越好, 理想情况下, $S_p = P$. 但实际上只有类似于向量加法等非常特殊的情况才能达到此完全加速比, 一般情况是 $S_p \leq P$.

与加速比有关的概念之一是并行算法的效率, 我们将其定义为 $E_p = S_p/P$. 有时候, 一个并行算法虽然有好的加速比, 但处理机的利用率可能很低, 特别是当处理机台数 $P(n)$ 不固定的情况下, S_p 不是一个最好的评价标准.

引入并行算法的效率后, 就可以度量并行系统中处理机能力发挥的程度. 显然在理想情况下 $E_p = 1$. 一个较现实的目标是构造具有对处理机台数 P 为线性加速比的算法, 即 $S_p \approx cP$, 其中 $0 < c \leq 1$ 与 P 无关, 其越接近于 1 越好.

影响并行算法加速比和效率的主要因素, 除了算法本身缺乏足够的并行度和数据通信外, 还有存取冲突和同步开销, 后两种情况都会引起若干台处理机的闲置.

2.6 并行算法的相容性和独立性

一个问题的两种算法是相容的, 是指问题的规模无限增大时, 两个算法中的运算量之比保持有界. 用数学语言描述就是: 当 $n \rightarrow \infty$ 时, $V(n)/S(n)$ 有界, 则称同一问题的一个向量或并行算法与串行算法是相容的; 否则, 称为不相容的. 其中 n 是问题的规模, $V(n)$ 和 $S(n)$ 分别为向量或并行算法和串行算法中的运算总数.

实际情况是同一问题的并行或向量算法与串行算法并非总相容. 例如, 通过修改串行算法设计出来的并行或向量算法, 其所含运算总数可能多于原串行算法运算总数, 即出现冗余运算. 于是就有可能发生当问题规模维持在一定限度之内时, 并行或向量算法比串行计算有利; 但是超出该限度时, 则其实际上变得比串行计算还坏. 原因就是冗余运算数量的不断增加, 从而加大在计算中的影响并达到转折点, 这就是一种不相容的情况.

需要说明的是, 不相容的算法未必没有实用价值. 人们熟知的递推倍增算法是非常有用的, 但它却是不相容的.

设计并行算法依赖于一个简单而重要的事实, 即独立的计算可以同时执行. 一组计算如果其每一个结果元仅在一次计算中出现, 则称为是独立的. 如果一个算法包含大量的独立计算, 我们说这个算法具有内在的并行性.

计算的独立性和相关性可进一步构成数值解法和算法的算法元之间的关系具体刻画, 简单算法元由下式定义:

$$y \leftarrow x_1 \circ x_2$$

其中

- ▶ x_1 和 x_2 是操作数, 称为输入元;
- ▶ $\circ$ 是某种算术运算, 如 $+$, $-$, $\times$, $/$ 或逻辑运算, 称为简单算子;
- ▶ y 是计算结果, 称为输出元.

如果由多个算法元组成的一组计算是独立的, 则这组计算的每个算法元的输出元都不是其余算法元的输入元. 反之, 如果存在一个算法元, 它的一个输入元是这组计算算法元的输出元, 那么这组计算就是相关计算.

一般而言, 由算子 φ 确定的算法元 A 的表示形式为:

$$A: (y_1, \dots, y_n) \leftarrow \varphi(x_1, \dots, x_r)$$

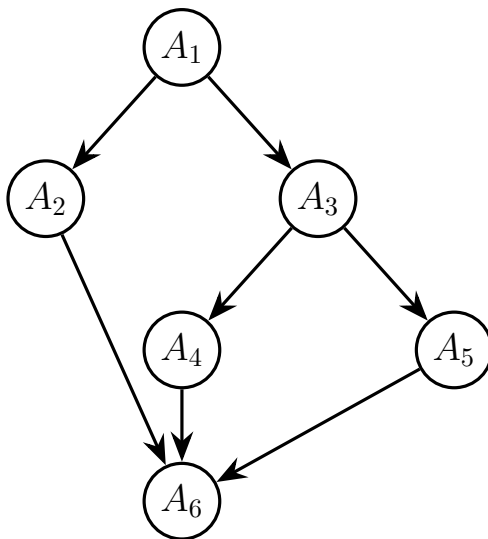
执行算法元 A 就是将输入元 $x_1, \dots, x_r$ 的值在算子下的像赋给输出元 $y_1, \dots, y_n$.

现设 W 是由算法元组成的有限集, 引入称为前趋关系的次序概念: 设 $A, B \in W$ 是两个算法元, 如果 B 的输入元含有 A 的输出元, 则称 A 是 B 的直接前趋, B 是 A 的直接后继. 如果存在 $C_i \in W, 1 \leq i \leq k, C_1 = A, C_k = B$ 使得 C_i 是 C_{i+1} 的直接前趋, 则称 A 是 B 的前趋, B 是 A 的后继, 记作 $A \prec B$, 并称序列 $\{C_1, \dots, C_k\}$ 是从 A 到 B 的一条长为 k 的路径.

A 不是 B 的前趋记作 $A \not\prec B$. 一般, 如果有 $A \prec B$ 或 $B \prec A$, 则称 A 与 B 前趋相关; 否则称 A 与 B 前趋无关或独立.

设 p 是一个求解问题, $(W, \prec)$ 是算法元的有序集, 如果按照任何保持前趋关系的次序执行 $(W, \prec)$ 中的算法元都能实现 p 的求解, 则称 $(W, \prec)$ 是 p 的一个解法.

次序可以是单枝或多枝的, 单枝次序依次执行单个算法元, 多枝次序则包含多个算法元的同时执行 (多枝情况反映了算法元计算的可并行性, 因为出现在多枝中的诸算法元是前趋无关或独立的). 可以利用由节点和定向连线组成的前趋图描绘给定的解法 $(W, \prec)$, 例如:



上图就是一个简单的前趋图, 根据图中所示前趋相关和无关的情况, 可以看出算法元 A_2 可以与 A_3 或算法元 A_4 、 A_5 同时计算.

现在考虑求解三角形方程组的并行算法, 方程组为 $Tx = b$, 为了叙述方便我们假设矩阵 T 是五阶的.

第一步是求解 $T_{11}x_1 = b_1$. x_1 解出后, 即可以用于右端项的校正:

$$b_2 = b_2 - T_{21}x_1; \quad b_3 = b_3 - T_{31}x_1$$

$$b_4 = b_4 - T_{41}x_1; \quad b_5 = b_5 - T_{51}x_1$$

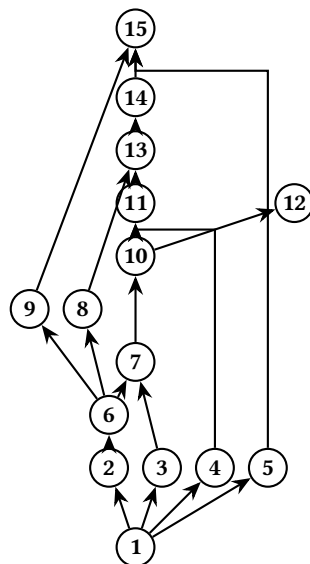
这四组计算具有无关性, 都是独立的计算, 可以并行计算.

一旦 b_2 得到校正, 即可计算 $x_2 = T_{22}^{-1}b_2$ (注意, 这里的计算又是与包含在 b_3, b_4, b_5 中的矩阵

向量计算无关). x_2 得到后, 又可校正右端项的当前值:

$$b_3 = b_3 - T_{32}x_2; \quad b_4 = b_4 - T_{42}x_2; \quad b_5 = b_5 - T_{52}x_2$$

如此继续下去, 直到得到最后解答. 下图是计算过程的前趋图:



能够实现并行计算的基础是计算的无关性, 而建立并行算法和发展并行数值分析的主要策略、途径和原则是分而治之. 从事并行计算研究和实践的人需要有很强的并行意识, 这种并行意识要求它们善于洞察和发现计算无关性, 要求他们善于把分而治之的普遍原则灵活而有效地应用于发展适合并行计算的新算法和新解法.

所谓分而治之的原则, 是把一个问题分裂为若干个较小的子问题, 这些问题可彼此独立地并行求解. 为了实现传统计算方法的并行化, 需要在如何“分”和“治”上花费力气. 分而治之的原则在比较复杂的一些问题上的应用是并行算法研究的基本内容, 目前已经产生了不少有意义的研究成果, 例如: 求解数学物理问题的区域分裂法、解三对角方程组的 Wang 算法、解线性方程组的多分裂迭代法、解抛物型偏微分方程的有限差分分段 (块) 隐式方法等, 都是成功地运用了分而治之原则的典型例子.

3 线性递推问题并行计算

递推问题呈现出高度的时序性或相关性, 适宜于 SISD 型计算机. 然而, 由于递推问题在实际应用中经常出现, 所以研究其的并行处理是并行算法的重要内容之一, 并且具有重要的实践意义.

早在 70 年代 Kogge 和 Stone 等就进行了研究, 取得了重要的成果, 其并行处理的一般方法是将递推关系式展开, 然后根据不同的展开式寻求不同的并行求值算法, 倍增法就是其中之一.

3.1 倍增法

考虑一阶线性递推关系式

$$\begin{cases} x_1 = b_1 \\ x_i = a_i x_{i-1} + b_i \quad i = 2, 3, \dots, n \end{cases}$$

这个问题似乎仅适合于串行计算, x_i 的计算依赖于前 $i-1$ 个量的计算, 前面的量没有算出来, 紧跟着的量则不能计算. 但注意到计算 x_i 所需的 $x_1, x_2, \dots, x_{i-1}$ 都是由 $a_1, \dots, a_{i-1}$ 和 $b_1, \dots, b_{i-1}$ 这些原始数据算出来的, 也就是说 x_i 是这些原始数据用乘法和加法运算结合在一起的一个算术表达式. 发现这些表达式的结构特点并利用其设计并行算法就是 Kogge 和 Stone 的工作最主要的点.

下面介绍他们建立的并行算法, 即倍增法.

设 $s, t \leq n$, 定义如下的函数关系式:

$$Q(s, t) = \sum_{j=s}^t \left(\prod_{r=j+1}^t a_r \right) b_j$$

其中, 当 $j \geq t$ 时, 规定 $\prod_{r=j+1}^t a_r = 1$. 可以证明 $Q(s, t)$ 有以下性质:

1. $Q(i, i) = b_i, i = 1, 2, \dots, n$.
2. 当 $s < t$, 对任意非负整数 $l, s+l < t$, 我们有

$$Q(s, t) = \left(\prod_{r=s+l+1}^t a_r \right) Q(s, s+l) + Q(s+l+1, t)$$

3. 由 (1) 和 (2) 用数学归纳法可证 $Q(1, i) = x_i, i = 1, 2, \dots, n$.

证明. (1) 和 (3) 是显然的. (2) 可由函数的定义直接得出:

$$\begin{aligned} Q(s, t) &= \sum_{j=s}^{s+l} \left(\prod_{r=j+1}^t a_r \right) b_j + \sum_{j=s+l+1}^t \left(\prod_{r=j+1}^t a_r \right) b_j \\ &= \left(\prod_{r=s+l+1}^t a_r \right) \sum_{j=s}^{s+l} \left(\prod_{r=j+1}^{s+l} a_r \right) b_j + Q(s+l+1, t) \\ &= \left(\prod_{r=s+l+1}^t a_r \right) Q(s, s+l) + Q(s+l+1, t) \end{aligned}$$

□

若令 $t = 2s$ 和 $t = 2s + 1$, 则可得:

$$\begin{cases} x_{2s} = (\prod_{r=s+1}^{2s})Q(1, s) + Q(s+1, 2s) \\ x_{2s+1} = (\prod_{r=s+1}^{2s+1} a_r)Q(1, s) + Q(s+1, 2s+1) \end{cases}$$

此即一阶线性递推问题倍增法计算公式.

倍增法的分解与计算过程如下: 不妨设 $n = 2^m$, 以计算 $x_n = Q(1, n)$ 为例, 有

$$x_n = Q(1, 2^m) = (\prod_{r=2^{m-1}+1}^{2^m} a_r)Q(1, 2^{m-1}) + Q(2^{m-1}+1, 2^m)$$

在此分解中, $Q(1, 2^{m-1})$ 与 $Q(2^{m-1}+1, 2^m)$ 结构完全相同, 可以同时计算. 对其又可继续递推地进行分解, 直至最简形式, 而且在计算时还可同时计算 $\prod a_r$.

如此分解进行到底便得到计算 x_n 的一个并行算法, 其执行次序正好和分解次序相反. 以 $n = 2^3$ 为例, 计算 x_8 的过程递推分解如下:

1. $Q(1, 8) = a_8 a_7 a_6 a_5 Q(1, 4) + Q(5, 8);$
2. $Q(1, 4) = a_4 a_3 Q(1, 2) + Q(3, 4), \quad Q(5, 8) = a_8 a_7 Q(5, 6) + Q(7, 8);$
3. $Q(1, 2) = a_2 Q(1, 1) + Q(2, 2), \quad Q(3, 4) = a_4 Q(3, 3) + Q(4, 4)$
 $Q(5, 6) = a_6 Q(5, 5) + Q(6, 6), \quad Q(7, 8) = a_8 Q(7, 7) + Q(8, 8);$

对于 x_7, x_6, x_5 可以类似地进行 3 次分解, 化为最简形式. 一般情况下, 使用 n 台处理器, 计算 n 个元素值 x_k , 需 $\lceil \log_2 n \rceil$ 步计算. 可以用状态图演示如下, 红色表示已算完的值:

表 1: 全局状态演进矩阵图 (Kogge-Stone 模式)

计算步骤	P_1	P_2	P_3	P_4	P_5	P_6	P_7	P_8
初始 (跨度 1)	[1, 1]	[2, 2]	[3, 3]	[4, 4]	[5, 5]	[6, 6]	[7, 7]	[8, 8]
第 1 步 (找前 1 个)	[1, 1]	[1, 2]	[2, 3]	[3, 4]	[4, 5]	[5, 6]	[6, 7]	[7, 8]
第 2 步 (找前 2 个)	[1, 1]	[1, 2]	[1, 3]	[1, 4]	[2, 5]	[3, 6]	[4, 7]	[5, 8]
第 3 步 (找前 4 个)	[1, 1]	[1, 2]	[1, 3]	[1, 4]	[1, 5]	[1, 6]	[1, 7]	[1, 8]

令

$$A_i = \begin{pmatrix} a_i & b_i \\ 0 & 1 \end{pmatrix}, \quad \mathbf{x}_i = \begin{pmatrix} x_i \\ 1 \end{pmatrix}$$

则原问题变成 $\mathbf{x}_i = A_i \mathbf{x}_{i-1}$, 故有 $\mathbf{x}_i = A_i \cdots A_2 \mathbf{x}_1$, 所以应用结合扇入法可以在 $\lceil \log_2 n \rceil$ 步算出所有的 x_i ($1 \leq i \leq n$).

下面给出阵列机情形的复杂性分析.

设一次乘法和一次加法所需的时间分别为 $\tau_{\times}$ 和 τ_{+} . 串行递推计算共需 $n - 1$ 步, 每一步都包含一次乘法和一次加法, 因而串行算法的运行时间为

$$T_1 = (\tau_{\times} + \tau_{+})(n - 1).$$

按倍增法组织并行计算时, 可分 $p = n$ 和 $p < n$ 两种情形讨论.

当 $p = n$ 时, 需要 $\lceil \log_2 n \rceil$ 个并行步. 在每一步中, 每个活跃处理机都要完成一次形如

$$(\alpha, \beta) \otimes (\gamma, \delta) = (\alpha\gamma, \beta\gamma + \delta)$$

的合并运算, 因此每步包含两次乘法和一次加法. 于是有

$$T_n = (2\tau_{\times} + \tau_{+})\lceil \log_2 n \rceil.$$

相应的加速比和效率分别为

$$S_n = \frac{T_1}{T_n} = \frac{(\tau_{\times} + \tau_{+})(n-1)}{(2\tau_{\times} + \tau_{+})\lceil \log_2 n \rceil},$$

$$E_n = \frac{S_n}{n} = \frac{(\tau_{\times} + \tau_{+})(n-1)}{n(2\tau_{\times} + \tau_{+})\lceil \log_2 n \rceil}.$$

当 $p < n$ 时, 可将全部数据分成 $\lceil \frac{n}{p} \rceil$ 轮, 每一轮在 p 台处理机上完成一次长度不超过 p 的倍增计算. 由于每一轮仍需 $\lceil \log_2 p \rceil$ 个并行步, 因而

$$T_p = (2\tau_{\times} + \tau_{+})\lceil \log_2 p \rceil \lceil \frac{n}{p} \rceil.$$

于是

$$S_p = \frac{T_1}{T_p} = \frac{(\tau_{\times} + \tau_{+})(n-1)}{(2\tau_{\times} + \tau_{+})\lceil \log_2 p \rceil \lceil \frac{n}{p} \rceil},$$

$$E_p = \frac{S_p}{p} = \frac{(\tau_{\times} + \tau_{+})(n-1)}{p(2\tau_{\times} + \tau_{+})\lceil \log_2 p \rceil \lceil \frac{n}{p} \rceil}.$$

这些公式表明, 倍增法将原问题中长度为 $n-1$ 的串行相关链压缩到了对数级深度. 当处理机数足够多时, 运行时间主要随 $\log_2 n$ 增长; 当处理机数受限时, 额外代价则主要体现为批次数 $\lceil \frac{n}{p} \rceil$ 的增加.

3.2 循环加倍法

对于

$$\begin{cases} x_1 = b_1, \\ x_i = a_i x_{i-1} + b_i, \quad i = 2, 3, \dots, n \end{cases}$$

可逐次用下述公式计算. 为方便起见, 设 $n = 2^m$.

$$x_i = a_i a_{i-1} x_{i-2} + a_i b_{i-1} + b_i = a_i^{(1)} x_{i-2} + b_i^{(1)},$$

其中

$$a_i^{(1)} = a_i a_{i-1}, \quad b_i^{(1)} = a_i b_{i-1} + b_i.$$

引入 $i = 1 + 2j$, 则有

$$x_{1+2j} = a_{1+2j}^{(1)} x_{1+2(j-1)} + b_{1+2j}^{(1)}, \quad j = 1, \dots, 2^{m-1} - 1,$$

其中

$$\begin{cases} a_{1+2j}^{(1)} = a_{1+2j} a_{2j}, \\ b_{1+2j}^{(1)} = a_{1+2j} b_{2j} + b_{1+2j}. \end{cases}$$

仿此推导可得

$$x_{1+2^2j} = a_{1+2^2j}^{(2)} x_{1+2^2(j-1)} + b_{1+2^2j}^{(2)}, \quad j = 1, \dots, 2^{m-2} - 1,$$

其中

$$\begin{cases} a_{1+2^2j}^{(2)} = a_{1+2^2j}^{(1)} a_{1+2(2j-1)}^{(1)}, \\ b_{1+2^2j}^{(2)} = a_{1+2^2j}^{(1)} b_{1+2(2j-1)}^{(1)} + b_{1+2^2j}^{(1)}. \end{cases}$$

一般地, 对 $k = 1, 2, \dots, m-1$, 有

$$x_{1+2^k j} = a_{1+2^k j}^{(k)} x_{1+2^k(j-1)} + b_{1+2^k j}^{(k)}, \quad j = 1, \dots, 2^{m-k} - 1,$$

其中

$$\begin{cases} a_{1+2^k j}^{(k)} = a_{1+2^k j}^{(k-1)} a_{1+2^{k-1}(2j-1)}^{(k-1)}, \\ b_{1+2^k j}^{(k)} = a_{1+2^k j}^{(k-1)} b_{1+2^{k-1}(2j-1)}^{(k-1)} + b_{1+2^k j}^{(k-1)}. \end{cases}$$

要实现原式并行计算, 首先要对上式按 k 方向串行、按 j 方向并行地组织计算.

下面以 $n = 16$ 为例说明循环加倍法的计算过程. 记 $\odot$ 表示按分量相乘.

当 $k = 1, j = 1, 2, \dots, 7$ 时, 有

$$\begin{aligned} \begin{bmatrix} a_3^{(1)} \\ a_5^{(1)} \\ \vdots \\ a_{15}^{(1)} \end{bmatrix} &= \begin{bmatrix} a_3 \\ a_5 \\ \vdots \\ a_{15} \end{bmatrix} \odot \begin{bmatrix} a_2 \\ a_4 \\ \vdots \\ a_{14} \end{bmatrix}, \\ \begin{bmatrix} b_3^{(1)} \\ b_5^{(1)} \\ \vdots \\ b_{15}^{(1)} \end{bmatrix} &= \begin{bmatrix} a_3 \\ a_5 \\ \vdots \\ a_{15} \end{bmatrix} \odot \begin{bmatrix} b_2 \\ b_4 \\ \vdots \\ b_{14} \end{bmatrix} + \begin{bmatrix} b_3 \\ b_5 \\ \vdots \\ b_{15} \end{bmatrix}. \end{aligned}$$

当 $k = 2, j = 1, 2, 3$ 时, 有

$$\begin{aligned} \begin{bmatrix} a_5^{(2)} \\ a_9^{(2)} \\ a_{13}^{(2)} \end{bmatrix} &= \begin{bmatrix} a_5^{(1)} \\ a_9^{(1)} \\ a_{13}^{(1)} \end{bmatrix} \odot \begin{bmatrix} a_3^{(1)} \\ a_7^{(1)} \\ a_{11}^{(1)} \end{bmatrix}, \\ \begin{bmatrix} b_5^{(2)} \\ b_9^{(2)} \\ b_{13}^{(2)} \end{bmatrix} &= \begin{bmatrix} a_5^{(1)} \\ a_9^{(1)} \\ a_{13}^{(1)} \end{bmatrix} \odot \begin{bmatrix} b_3^{(1)} \\ b_7^{(1)} \\ b_{11}^{(1)} \end{bmatrix} + \begin{bmatrix} b_5^{(1)} \\ b_9^{(1)} \\ b_{13}^{(1)} \end{bmatrix}. \end{aligned}$$

当 $k = 3, j = 1$ 时, 有

$$a_9^{(3)} = a_9^{(2)} a_5^{(2)}, \quad b_9^{(3)} = a_9^{(2)} b_5^{(2)} + b_9^{(2)}.$$

计算出 $a_{1+2^k j}^{(k)}$ 和 $b_{1+2^k j}^{(k)}$ 以后, 再回代求解 $x_{1+2^k j}$, 即先计算 x_9 , 最后计算 $x_2, x_4, \dots, x_{16}$.

当 $k = 3, j = 1$ 时, 有

$$x_9 = a_9^{(3)} x_1 + b_9^{(3)}.$$

当 $k = 2, j = 1, 3$ 时, 有

$$\begin{bmatrix} x_5 \\ x_{13} \end{bmatrix} = \begin{bmatrix} a_5^{(2)} & 0 \\ 0 & a_{13}^{(2)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_9 \end{bmatrix} + \begin{bmatrix} b_5^{(2)} \\ b_{13}^{(2)} \end{bmatrix}.$$

当 $k = 1, j = 1, 3, 5, 7$ 时, 有

$$\begin{bmatrix} x_3 \\ x_7 \\ x_{11} \\ x_{15} \end{bmatrix} = \begin{bmatrix} a_3^{(1)} & 0 & 0 & 0 \\ 0 & a_7^{(1)} & 0 & 0 \\ 0 & 0 & a_{11}^{(1)} & 0 \\ 0 & 0 & 0 & a_{15}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_5 \\ x_9 \\ x_{13} \end{bmatrix} + \begin{bmatrix} b_3^{(1)} \\ b_7^{(1)} \\ b_{11}^{(1)} \\ b_{15}^{(1)} \end{bmatrix}.$$

当 $k = 0, j = 1, 3, 5, 7, 9, 11, 13, 15$ 时, 有

$$\begin{bmatrix} x_2 \\ x_4 \\ \vdots \\ x_{16} \end{bmatrix} = \begin{bmatrix} a_2 & 0 & \cdots & 0 \\ 0 & a_4 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{16} \end{bmatrix} \begin{bmatrix} x_1 \\ x_3 \\ \vdots \\ x_{15} \end{bmatrix} + \begin{bmatrix} b_2 \\ b_4 \\ \vdots \\ b_{16} \end{bmatrix}.$$

下面分析循环加倍法的复杂性, 仅对 $P = n$ 情况进行讨论; 当 $P < n$ 时, 可分 $n = 2^r P$ 和 $n < 2^r P$ 两种情况进行讨论.

- ▶ 当 $P = n$ 时, 先按 j 方向并行, k 方向串行组织计算.
- ▶ 由于要加工的 $\{a_{1+2^k j}^{(k)}\}$ 和 $\{b_{1+2^k j}^{(k)}\}$ 个数均小于或等于 $n/2$, 所以可以同时计算, 且

$$k = 1, 2, \dots, (\lceil \log_2 n \rceil - 1).$$

这样加工 $\{a_{1+2^k j}^{(k)}\}$ 和 $\{b_{1+2^k j}^{(k)}\}$ 需 $\lceil \log_2 n \rceil - 1$ 步, 每步一个乘法和一个加法, 这部分计算时间为

$$T_2^{(1)} = (\tau_{\times} + \tau_{+})(\lceil \log_2 n \rceil - 1).$$

- ▶ 其次按 j 方向并行, k 方向串行组织计算. 计算所有 $\{x_{1+2^k j}\}$ 共需 $\lceil \log_2 n \rceil$ 步, 每步一个乘法和一个加法, 这部分计算时间为

$$T_2^{(2)} = (\tau_{\times} + \tau_{+})\lceil \log_2 n \rceil.$$

所以 $P = n$ 时, 循环加倍法的运行时间、加速比和效率分别为

$$\begin{cases} t_c = T_2^{(1)} + T_2^{(2)} = (\tau_{\times} + \tau_{+})(2\lceil \log_2 n \rceil - 1), \\ S_p = \frac{(\tau_{\times} + \tau_{+})(n - 1)}{(\tau_{\times} + \tau_{+})(2\lceil \log_2 n \rceil - 1)} = \frac{n - 1}{2\lceil \log_2 n \rceil - 1}, \\ E_p = \frac{(\tau_{\times} + \tau_{+})(n - 1)}{P(\tau_{\times} + \tau_{+})(2\lceil \log_2 n \rceil - 1)} = \frac{n - 1}{P(2\lceil \log_2 n \rceil - 1)}. \end{cases}$$

特别地, 当 $P = n$ 时,

$$E_p = \frac{n - 1}{n(2\lceil \log_2 n \rceil - 1)}.$$

4 并行矩阵-向量乘法

设 A 为 $m \times n$ 矩阵, x 为 n 维向量, 则

$$Ax = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{bmatrix}.$$

我们以 a_i 和 a^j 分别表示矩阵 A 的第 i 个行向量和第 j 个列向量, 以 a_{ij} 表示 A 的第 i 行第 j 列处的元素. 完成此式的计算可有两途径.

4.1 两种计算方式

第一种方法是建立在向量内积计算的基础上. 因为

$$Ax = \begin{bmatrix} (a_1, x) \\ (a_2, x) \\ \vdots \\ (a_m, x) \end{bmatrix},$$

其中

$$(x, y) = \sum_{i=1}^n x_i y_i$$

为普通向量内积, 所以矩阵-向量乘法 Ax 要求做 m 次向量内积计算.

另一种观点是把 Ax 看作 A 的列向量的线性组合, 即

$$Ax = \sum_{j=1}^n x_j a^j.$$

这两种方法的不同点可以从下面的计算程序看出:

```

y ← 0
for i = 1 to m do
  for j = 1 to n do
    y_i ← y_i + a_ij x_j
  end for
end for

```

```

y ← 0
for j = 1 to n do
  for i = 1 to m do
    y_i ← y_i + a_ij x_j
  end for
end for

```

左边程序对每个 i 做关于 j 的循环, 实现 A 的第 i 行和 x 的内积计算; 右边程序对应着线性组合的方式. 注意到两段程序最后的算术计算语句是相同的, 区别只在于循环次序不同.

4.2 向量计算分析

对上述算法的向量和并行计算做简单分析.

1. 向量计算方式要求的内积计算过程为

$$t_i = x_i y_i, \quad i = 1, \dots, n, \quad (x, y) = \sum_{i=1}^n t_i.$$

第一步是一个向量-向量乘法, 第二步是求和, 可由扇入法计算. 在向量计算机上执行时, 其求和向量长度将逐次减半, 而且当向量长度减小到一定限度时, 后续过程往往不如标量计算更有效.

2. 线性组合计算方式可写为

```

y ← 0
for j = 1 to n do
    y ← y + x_j a^j
end for

```

其中 x_j 为标量, a^j 为向量, 所以每一步计算都是向量加上标量与向量的乘积. 有些向量计算机, 如 Cyber 205, 对处理这种“向量 + 标量 × 向量”形式的计算特别有效, 其运算速度与向量加法几乎相同. 这种形式的计算通常称为链三元组 (Linked Triad).

4.3 并行计算分析

先考虑线性组合计算方式. 为简单起见, 假定并行计算机所含处理机台数 $P = n$. 计算安排如下: 第 i 台处理机 P_i 作 x_i 与 a^i 的乘积计算, 使得式中所含乘积 $x_i a^i$ 的计算具有理想的并行度.

但是, 为了得到最后结果向量还需要做加法运算, 而加法运算可由对向量求和的算法来完成, 其并行度相对稍差. 另外, 如果图中所示数据是局部存储的, 那么在做这些加法计算时, 各处理机之间还存在数据传输问题; 如果采用的是共享存储类型计算机, 则没有数据通信问题, 图中所示只是任务划分, 所有数据都存于共享内存之中.

在一定情况下, 内积算法有明显优势. 设 $P = m$, a_i 为 A 的第 i 行, 把 x 和 a_i 分配给第 i 台处理机. 这样每台处理机只需做一个内积计算, 因而可有最理想的并行度 m , 既不用计算扇入加法, 也不需要额外作数据传输.

4.4 实际选型讨论

在实际应用中选用哪一种计算方法, 还需要考虑具体情况.

- ▶ 一般说来, 矩阵-向量乘法运算只是作为计算的一部分出现在一个较大规模的计算问题之中.
- ▶ 假如采用的是具有局部存储的多处理机, 并且如果 x_i 和列向量 a^i 已经存储在第 i 台处理机中, 自然要选取线性组合方法, 尽管其计算的并行度较差.

- ▶ 同时也应注意到, 矩阵-向量计算的最后结果保存在不同处理机中. 如果还要把这个结果继续作为向量使用, 这种分散存放的方式会带来一些不便.
- ▶ 前面的分析默认处理机台数恰好等于矩阵 A 的行数或列数, 这是一种理想化情形. 实际中常常出现 m 和 n 远大于处理机台数 P 的情况, 此时便需要进一步考虑矩阵分块计算方法, 相应的算法分析也会更加复杂.

5 矩阵乘积并行计算

从矩阵乘积定义出发, 按照常规算法做矩阵乘积虽然结构简单, 但运算量却很大. 对于 n 阶方阵乘积, 串行计算的运算量为

$$n^2(2n - 1).$$

下面先讨论按照定义进行矩阵乘积的几种并行处理方式, 然后介绍两种快速矩阵乘法: Strassen 方法和 Winograd 方法.

5.1 内积、中积与外积算法

设 A 和 B 分别为 $m \times n$ 和 $n \times q$ 矩阵, 则积

$$C = AB$$

是一个 $m \times q$ 矩阵. 下面介绍三种主要计算方法.

5.1.1 内积算法

内积算法是矩阵-向量乘法内积形式的自然推广. 若把 A 划分为行向量

$$A = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix},$$

把 B 划分为列向量

$$B = (b^1, b^2, \dots, b^q),$$

则矩阵乘积的每个元素都由一个内积给出:

$$C = AB = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix} (b^1, b^2, \dots, b^q) = ((a_i, b^j))_{m \times q}.$$

当 $q = 1$ 时, 上式就退化为矩阵-向量乘法的内积算法. 若用 n 台处理机计算某个元素

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj},$$

则可先并行计算 $a_{ik} b_{kj}$ ($k = 1, 2, \dots, n$), 再利用扇入法做 $\lceil \log_2 n \rceil$ 个加法步骤求得 c_{ij} . 如果共有 mnq 台处理机, 那么全部矩阵乘法运算共需

$$1 + \lceil \log_2 n \rceil$$

步.

从并行复杂性上看, 内积算法是很理想的; 遗憾的是所需处理机台数太多, 实际运算中一般不可能满足, 同时这里还没有计入数据传输时间. 对于流水线向量机, 内积算法还会引起向量操作数相关, 从而使数据调度较为困难.

5.1.2 中积算法

若把矩阵乘法建立在重复应用矩阵-向量乘法的基础上, 令 a^j 表示 A 的第 j 列, b^i 表示 B 的第 i 列, 则

$$C = AB = (Ab^1, Ab^2, \dots, Ab^q).$$

进一步展开可得

$$C = \left(\sum_{j=1}^n b_{j1}a^j, \sum_{j=1}^n b_{j2}a^j, \dots, \sum_{j=1}^n b_{jq}a^j \right).$$

其中 C 的第 i 列恰好是 A 乘 B 的第 i 列 b^i , 从而把 AB 的计算变成了 q 个矩阵-向量乘积的运算; 而上式最后一步又正对应于矩阵-向量乘法的线性组合算法.

下图描述的程序称为中积算法:

```

C ← 0
for i = 1 to q do
    for j = 1 to n do
        ci ← ci + bjiaj
    end for
end for

```

5.1.3 外积算法

设 u 为长度为 m 的列向量, v 为长度为 q 的行向量, 则 u 与 v 的外积定义为

$$uv = (v_1u, v_2u, \dots, v_qu).$$

将 A 写成列分块形式

$$A = (a^1, a^2, \dots, a^n),$$

将 B 写成按行堆叠的形式

$$B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix},$$

其中 b_i 为 B 的第 i 行, 则矩阵乘法可写为

$$C = AB = (a^1, a^2, \dots, a^n) \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = \sum_{i=1}^n a^i b_i.$$

若再把 $b_i = (b_{i1}, b_{i2}, \dots, b_{iq})$, 则

$$C = \sum_{i=1}^n (b_{i1}a^i, b_{i2}a^i, \dots, b_{iq}a^i).$$

外积算法的程序表示为

```

C ← 0
for i = 1 to n do
  for j = 1 to q do
    cj ← cj + bijai
  end for
end for

```

5.1.4 三种算法的比较

为了更直观地说明三种算法, 取

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}.$$

则

$$C = AB = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} & a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} & a_{21}b_{13} + a_{22}b_{23} + a_{23}b_{33} \end{bmatrix}.$$

内积算法直接由矩阵乘法定义给出, 即 C 的每个元素都由 A 的行向量与 B 的列向量作内积来确定.

中积算法则把 C 写成按列计算的形式:

$$C = \begin{bmatrix} b_{11} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} + b_{21} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} + b_{31} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} \\ b_{12} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} + b_{22} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} + b_{32} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} \\ b_{13} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} + b_{23} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} + b_{33} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} \end{bmatrix}.$$

外积算法则把求和运算拿到矩阵之外:

$$C = \begin{bmatrix} b_{11} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} & b_{12} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} & b_{13} \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} \\ + \begin{bmatrix} b_{21} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} & b_{22} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} & b_{23} \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} \\ + \begin{bmatrix} b_{31} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} & b_{32} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} & b_{33} \begin{bmatrix} a_{13} \\ a_{23} \end{bmatrix} \end{bmatrix}.$$

类似上述差异, 矩阵相乘不同算法的差异也反映在下标 i, j, k 的不同次序上. 对

$$c_{ij} = \sum_k a_{ik} b_{kj}$$

而言, 下标 i, j, k 共有 6 种不同排列的可能, 对应着 6 个矩阵乘法算法, 这就是所谓的矩阵乘法的 ijk 形式, 上面提到的 3 个算法只是其中的 3 种情形.

在中积算法和外积算法中都包含了大量的链三元组计算, 这对于 Cyber 205 这类向量计算机

是很有效的. 特别是在外积算法中, 向量 a^i 被重复使用 q 次, 这对于 Cray 这类具有向量寄存器的机器来说是有利的, 因为参加运算的向量操作数能在寄存器中停留较久, 从而减少对主存储器的访问次数, 节省通信时间.

5.2 Strassen 算法

前面所讨论的矩阵乘法是根据定义, 对其算法结构做一些组合后讨论并行处理. 下面介绍由 V. Strassen 提出的快速矩阵乘法, 其基本思想是将二阶矩阵乘法所需的乘法运算由 8 个减少为 7 个.

对于阶为 2 的幂次的矩阵, 即 $n = 2^m$, 利用行列分裂技术, 先将矩阵分解成 2×2 的矩阵块, 每个块都是 $(n/2) \times (n/2)$ 阶矩阵:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}.$$

若记

$$C = AB = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix},$$

则按定义计算需要 8 次矩阵乘法与 4 次矩阵加法:

$$\begin{cases} C_{11} = A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} = A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} = A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} = A_{21}B_{12} + A_{22}B_{22}. \end{cases}$$

注意到上式关于下标 1, 2 具有对称性, 即通过下标 1, 2 的对调, C_{11} 与 C_{22} 、 C_{12} 与 C_{21} 可以相互生成. 为减少乘法次数, Strassen 先对 C_{12} 与 C_{21} 配置中间项, 令

$$\begin{cases} M_1 = A_{11}(B_{12} - B_{22}), \\ M_2 = (A_{11} + A_{12})B_{22}, \\ M_3 = A_{22}(B_{21} - B_{11}), \\ M_4 = (A_{21} + A_{22})B_{11}, \end{cases}$$

于是

$$C_{12} = M_1 + M_2, \quad C_{21} = M_3 + M_4.$$

同时又要求 C_{11} 与 C_{22} 的计算公式中包含关于下标 1, 2 对称的公共项, 设取

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22}),$$

则经过整理可得

$$C_{11} - M_5 = M_6 + M_3 - M_2,$$

其中

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22}),$$

而利用对称性还有

$$C_{22} - M_5 = M_7 + M_1 - M_4,$$

其中

$$M_7 = (A_{21} - A_{11})(B_{11} + B_{12}).$$

从而得到只用 7 次乘法的二阶矩阵乘法方案:

$$\begin{cases} M_1 = A_{11}(B_{12} - B_{22}), \\ M_2 = (A_{11} + A_{12})B_{22}, \\ M_3 = A_{22}(B_{21} - B_{11}), \\ M_4 = (A_{21} + A_{22})B_{11}, \\ M_5 = (A_{11} + A_{22})(B_{11} + B_{22}), \\ M_6 = (A_{12} - A_{22})(B_{21} + B_{22}), \\ M_7 = (A_{21} - A_{11})(B_{11} + B_{12}), \end{cases}$$

再按

$$\begin{cases} C_{11} = M_5 + M_3 - M_2 + M_6, \\ C_{12} = M_1 + M_2, \\ C_{21} = M_3 + M_4, \\ C_{22} = M_5 + M_1 - M_4 + M_7 \end{cases}$$

组成矩阵乘积 AB 的各个块. 这样共需 7 次乘法与 18 次加法, 比常规方法减少了 1 次乘法, 但增加了 14 次加法.

初看起来, Strassen 算法似乎并没有多少好处. 但若对上述 7 个阶数为 2^{m-1} 的矩阵乘积继续作行列分裂, 再递归使用 Strassen 算法, 一直分到二阶矩阵乘积为止, 其总运算量将显著下降.

设 $T(m)$ 与 $A(m)$ 分别表示两个 $n = 2^m$ 阶矩阵相乘所需的乘法、加法运算量. 由 $T(0) = 1$, $A(0) = 0$ 可得

$$T(m) = 7T(m-1) = 7^2T(m-2) = \cdots = 7^m = n^{\log_2 7} \approx n^{2.8074},$$

$$A(m) = 7A(m-1) + 18(2^{m-1})^2 = 6(7^m - 4^m) < 6 \cdot 7^m \approx 6n^{2.8074}.$$

因此 Strassen 算法把乘法和加法次数的运算量都降为

$$O(n^{2.8074}).$$

又由于

$$\lim_{n \rightarrow \infty} \frac{n^{2.8074}}{n^3} = 0,$$

所以它是一种渐近快速算法, 并且随着 n 增大, 其效果会越来越好.

若 n 不是 2 的幂次, 取 m 满足

$$2^{m-1} < n < 2^m,$$

先把 A, B 扩充为 2^m 阶矩阵, 再使用 Strassen 算法作矩阵乘积.

下面简单分析多大规模时 Strassen 算法优于常规乘法. 设常规矩阵乘法运算总量为

$$n^2(2n - 1).$$

当

$$n^2(2n - 1) \leq 7 \left(\frac{n}{2}\right)^2 \left(2 \cdot \frac{n}{2} - 1\right) + 18 \left(\frac{n}{2}\right)^2$$

时, 一层 Strassen 分解的运算量反而不如常规方法更小. 由此可得当 $n \leq 15$ 时, Strassen 算法运算量反而比常规乘法多.

对于流水线向量机, 现假定矩阵乘积采用外积算法, 设 σ 表示向量流水线机运算的启动时间, 则 Strassen 算法有效的矩阵阶数下界可由

$$n(2n - 1)(\sigma + n) \leq 7 \frac{n}{2} \left(2 \cdot \frac{n}{2} - 1\right) \left(\sigma + \frac{n}{2}\right) + 18 \frac{n}{2} \left(\sigma + \frac{n}{2}\right)$$

估计. 为简单起见, 若向量乘法与加法的启动时间都取 $\sigma = 5$, 则可算得在 $n \leq 48$ 时外积算法比 Strassen 算法更有效, 因而只有在 $n > 45$ 时, 使用 Strassen 算法才较为合算.

5.3 Winograd 算法

与 Strassen 算法相仿, Winograd 又提出了二阶矩阵乘法的另一方案. 仍考虑

$$C = AB = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}.$$

先令

$$M_1 = A_{11}B_{11}, \quad M_2 = A_{12}B_{21},$$

则

$$C_{11} = M_1 + M_2.$$

接着希望把 C_{12}, C_{21}, C_{22} 写成含有某个公共项 M_3 的形式. Winograd 的一种选取是

$$M_3 = (A_{21} + A_{22} - A_{11})(B_{22} + B_{11} - B_{12}).$$

于是有

$$M_1 + M_3 = C_{12} - M_4 - M_5,$$

其中

$$M_4 = (A_{11} + A_{12} - A_{21} - A_{22})B_{22}, \quad M_5 = (A_{21} + A_{22})(B_{12} - B_{11}).$$

又为得到 C_{21} , 在右端配上 $A_{22}B_{21}$ 项, 可得

$$M_1 + M_3 = C_{21} + M_6 - M_7,$$

其中

$$M_6 = A_{22}(B_{22} + B_{11} - B_{12} - B_{21}), \quad M_7 = (A_{11} - A_{21})(B_{22} - B_{12}).$$

而为了得到 C_{22} , 在右端配上 $A_{21}B_{12}$ 项, 有

$$M_1 + M_3 = C_{22} - M_5 - M_7.$$

由此可得 Winograd 算法的一个较紧凑写法:

$$\begin{aligned}
S_1 &= A_{21} + A_{22}, & S_2 &= S_1 - A_{11}, & S_3 &= A_{11} - A_{21}, & S_4 &= A_{12} - S_2, \\
S_5 &= B_{12} - B_{11}, & S_6 &= B_{22} - S_5, & S_7 &= B_{22} - B_{12}, & S_8 &= S_6 - B_{21}, \\
M_1 &= A_{11}B_{11}, & M_2 &= A_{12}B_{21}, & M_3 &= S_2S_6, & M_4 &= S_4B_{22}, \\
M_5 &= S_1S_5, & M_6 &= A_{22}S_8, & M_7 &= S_3S_7, \\
T_1 &= M_1 + M_3, & T_2 &= T_1 + M_4, & T_3 &= T_1 + M_7, \\
C_{11} &= M_1 + M_2, & C_{12} &= T_2 + M_5, & C_{21} &= T_3 - M_6, & C_{22} &= T_3 + M_5.
\end{aligned}$$

此算法共需 7 次乘法和 15 次加法, 加法次数比 Strassen 算法有所改进, 但乘法次数并未减少. 同样可以分析 Winograd 算法有效的矩阵阶数下界. 在纯标量运算量意义下, 若

$$n^2(2n-1) \leq 7 \left(\frac{n}{2}\right)^2 \left(2 \cdot \frac{n}{2} - 1\right) + 15 \left(\frac{n}{2}\right)^2,$$

则一层 Winograd 分解还不如常规算法合算. 由此得到当 $n \leq 12$ 时, 使用 Winograd 算法是不合算的.

对于并行流水线向量机, 若矩阵乘积采用外积算法, 则其有效阶数下界可由

$$n(2n-1)(\sigma+n) \leq 7 \frac{n}{2} \left(2 \cdot \frac{n}{2} - 1\right) \left(\sigma + \frac{n}{2}\right) + 15 \frac{n}{2} \left(\sigma + \frac{n}{2}\right)$$

估计. 与前面相同, 若取 $\sigma = 5$, 则可求得 $n \leq 44$ 时采用外积算法更有效, 因而只有在 $n > 45$ 时, 使用 Winograd 算法才较为合算.

6 并行矩阵求逆

矩阵求逆的有效算法通常也适用于求解线性方程组。下面介绍一类一般稠密矩阵求逆的消去法, 以及由分裂技术导出的分块递推矩阵求逆法。

6.1 一般稠密矩阵求逆的消去法

设可逆矩阵 $A \in \mathbb{R}^{n \times n}$. 求 A^{-1} 的消去法是一种较通用的方法. 其基本思想是寻找初等矩阵

$$P_1, P_2, \dots, P_m,$$

使得

$$P_m P_{m-1} \cdots P_1 A = I,$$

从而

$$A^{-1} = P_m P_{m-1} \cdots P_1.$$

若作 A 的 $n \times 2n$ 阶增广矩阵

$$B = (A, I),$$

则有

$$P_m P_{m-1} \cdots P_1 B = (I, A^{-1}).$$

因此, 只需对增广矩阵 B 施行 Gauss-Jordan 消去, 即可在右半部分得到 A^{-1} .

若记第 k 步消去后的增广矩阵元素为 $a_{ij}^{(k)}$, 则在不选主元的情况下, 对于

$$k = 1, 2, \dots, n,$$

先将第 k 行按主元 $a_{kk}^{(k-1)}$ 归一化, 再对其余各行消去第 k 列元素, 其更新公式可写成

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)} a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}},$$

其中

$$i = 1, \dots, k-1, k+1, \dots, n, \quad j = k+1, \dots, n+k.$$

这里把 slide 中容易混淆的列下标写成了 $j = k+1, \dots, n+k$, 因为增广矩阵共有 $2n$ 列, 而前 k 列在第 k 步之后已经不再需要更新。

6.1.1 Marshall 格式

下面介绍 Marshall 提出的一种尽量节省主存的计算格式. 其基本公式仍与一般消去法相同, 但存储组织方式不同。

设 R, S 为两个辅助存储器, 每个可存放 $n+1$ 个数; 主存中只设置一个可存放 $n \times (n+1)$ 个数的存储区. 将矩阵 A 存放在前 n 列, 其余一列按当前消去步需要临时存放增广矩阵右半部分的数据, 可以看成是压缩存放的增广矩阵. 最终在原矩阵 A 的位置上得到 A^{-1} .

为简明起见, 先讨论不选主元的情形. 对每个

$$k = 1, 2, \dots, n,$$

算法步骤如下:

1. 将压缩增广矩阵的第 k 行读入辅助存储器 R .
2. 计算主元倒数 $1/a_{kk}$.
3. 将向量 R 乘以 $1/a_{kk}$, 并仍保留在 R 中.
4. 对每个 $i \neq k$, 读出第 i 行的第 k 个元素 a_{ik} .
5. 计算 $-a_{ik}R \Rightarrow S$, 且不改变 R .
6. 把 S 中第 $n+1$ 个数加到第 k 个位置上, 然后将第 $n+1$ 个数置零.
7. 将向量 S 加到主存中第 i 行. 这一步的效果是在 A 中消去 a_{ik} , 同时把增广矩阵右半部分的相应元素写入该位置.
8. 重复步骤 4-7 直到所有 $i \neq k$ 都处理完毕. 最后把 R 中第 $n+1$ 个数放到第 k 个位置.
9. 将向量 R 送回主存第 k 行并覆盖原内容. 这样就把第 k 个对角元化为 1, 并把该列其他元素消为 0.
10. 若 $k < n$, 则令 $k \leftarrow k+1$ 并继续下一步消去; 当执行到 $k = n$ 时, 主存储器的前 n 列即为 A^{-1} .

若设并行机台数为 $n+1$, 则步骤 2、3、6 各需 n 步, 步骤 5、7 各需 $n(n-1)$ 步. 因而整个算法的并行运算步数约为

$$2n^2 + n,$$

而串行运算量约为

$$n(2n^2 - 2n + 1).$$

于是其加速比与效率分别满足

$$S_p = O(n), \quad E_p = O(1).$$

这种格式在节省存储空间的同时, 增加了一些读写与传送操作, 因此运算速度会受到一定影响. 对于存储器比较充足, 或矩阵阶数不太大的机器, 使用通常的增广矩阵消去法往往更合适.

6.2 分块递推法

对于某些特殊结构矩阵, 通过分块可以把求逆过程递归化. 下面先给出一般分块公式, 再讨论它对上三角矩阵求逆时的高效性.

为叙述方便, 设 n 为偶数, 并把矩阵分块为

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

其中每个子块都是 $n/2$ 阶矩阵.

若 A_{11} 可逆, 则有分解

$$\begin{bmatrix} I_{n/2} & 0 \\ -A_{21}A_{11}^{-1} & I_{n/2} \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} I_{n/2} & -A_{11}^{-1}A_{12} \\ 0 & I_{n/2} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

因此

$$A = \begin{bmatrix} I_{n/2} & 0 \\ A_{21}A_{11}^{-1} & I_{n/2} \end{bmatrix} \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix} \begin{bmatrix} I_{n/2} & A_{11}^{-1}A_{12} \\ 0 & I_{n/2} \end{bmatrix}.$$

若再记

$$A^{-1} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix},$$

则由上式可得

$$\begin{cases} C_{22} = (A_{22} - A_{21}A_{11}^{-1}A_{12})^{-1}, \\ C_{21} = -C_{22}A_{21}A_{11}^{-1}, \\ C_{12} = -A_{11}^{-1}A_{12}C_{22}, \\ C_{11} = A_{11}^{-1} - A_{11}^{-1}A_{12}C_{21}. \end{cases}$$

把它拆成可执行的步骤, 可以写成

$$\begin{aligned} B_1 &= A_{11}^{-1}, \\ B_2 &= A_{21}B_1, & B_3 &= B_1A_{12}, \\ B_4 &= A_{21}B_3, \\ B_5 &= A_{22} - B_4, \\ C_{22} &= B_5^{-1}, \\ C_{21} &= -C_{22}B_2, & C_{12} &= -B_3C_{22}, \\ B_6 &= B_3C_{21}, \\ C_{11} &= B_1 - B_6. \end{aligned}$$

于是求逆问题被归结为两个 $n/2$ 阶矩阵的求逆, 加上若干个 $n/2$ 阶矩阵乘法与加减法. 如果继续对 A_{11}^{-1} 与 B_5^{-1} 递归地使用同样方法, 则可不断分裂下去, 直到子块规模足够小而容易直接求解.

若矩阵乘法采用 Strassen 算法实现, 设 $T(n)$ 与 $T_1(n)$ 分别表示 n 阶矩阵乘法与 n 阶矩阵求逆的乘、加运算总次数, 则有递推关系

$$T_1(n) = 2T_1\left(\frac{n}{2}\right) + 6T\left(\frac{n}{2}\right) + 2\left(\frac{n}{2}\right)^2.$$

由此可得

$$T_1(n) < 8.6 n^{\log_2 7} \approx 8.6 n^{2.8074},$$

因此在乘法使用快速算法时, 分块递推求逆也具有相应的渐近快速性.

需要注意的是, 这种分块递推算法要求

$$A_{11}, \quad A_{22} - A_{21}A_{11}^{-1}A_{12}$$

都可逆, 这在一定程度上限制了它对一般矩阵求逆的通用性.

6.2.1 上三角矩阵的情形

不过对于三角矩阵求逆, 分块递推法会变得很高效. 设 A 为上三角矩阵,

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix},$$

其中 A_{11}, A_{22} 是两个 $n/2$ 阶上三角矩阵, 则

$$A^{-1} = \begin{bmatrix} A_{11}^{-1} & -A_{11}^{-1}A_{12}A_{22}^{-1} \\ 0 & A_{22}^{-1} \end{bmatrix}.$$

于是分裂一次后的计算步骤可以写成

- (1) 计算 A_{11}^{-1}, A_{22}^{-1} ,
- (2) 计算 $B_1 = -A_{11}^{-1}A_{12}$,
- (3) 计算 $B_2 = B_1A_{22}^{-1}$.

由此可见, n 阶上三角矩阵的求逆可以化为两个 $n/2$ 阶上三角矩阵的求逆, 外加两次 $n/2$ 阶矩阵乘法. 并且 A_{11}, A_{22} 仍然是上三角矩阵, 因而可以继续递归分裂, 直到达到最简规模为止.

7 线性方程组的并行求解

线性代数方程组的数值求解在科学计算与工程计算中应用非常广泛。虽然线性代数问题的计算方法和数值软件已比较成熟,但随着并行计算机与向量计算机的出现,线性方程组的有效并行求解仍然受到普遍关注。

大量研究工作一方面致力于开发传统算法的并行性,使之适应并行计算机结构,如 Gauss 消去法、Jacobi 迭代法等;另一方面则致力于研制新的有效并行解法,如三角方程组的分裂法、多分裂迭代法等。

7.1 三角方程组的并行求解

三角方程组是线性方程组数值求解中的一个基本问题。一般形式方程组的许多直接解法,最终都要归结为三角方程组的求解。下面以单位下三角或一般下三角方程组

$$Lx = b$$

为例进行讨论,其中

$$L = (l_{ij})_{n \times n}, \quad l_{ij} = 0 \ (i < j), \quad l_{ii} \neq 0.$$

对上三角方程组可以作完全类似的处理。

显然,上式求解可以化为如下递推问题:

$$\begin{cases} x_1 = b_1/l_{11}, \\ x_i = (b_i - \sum_{j=1}^{i-1} l_{ij}x_j)/l_{ii}, \quad i = 2, 3, \dots, n. \end{cases}$$

因此可以直接利用第二章中处理递推问题的方法组织并行计算。下面结合线性方程组求解的特点,给出几种更有针对性的并行算法。

7.1.1 行扫描法与列扫描法

考察上面的递推式,一个直接的组织方式是逐行组织并行计算,这就是所谓的行扫描算法,也常被称为内积算法。

算法 3.1 行扫描算法

```

 $x_1 \leftarrow b_1/l_{11}$ 
for  $i = 2$  to  $n$  do
    for  $j = 1$  to  $i - 1$  do
         $b_i \leftarrow b_i - l_{ij}x_j$ 
    end for
     $x_i \leftarrow b_i/l_{ii}$ 
end for

```

作为另一种替换方案,也可以按列组织并行计算,得到所谓的列扫描算法:

算法 3.2 列扫描算法

```

for  $j = 1$  to  $n - 1$  do
     $x_j \leftarrow b_j / l_{jj}$ 
    for  $i = j + 1$  to  $n$  in parallel do
         $b_i \leftarrow b_i - l_{ij} x_j$ 
    end for
end for
 $x_n \leftarrow b_n / l_{nn}$ 

```

这里把 slide 中行扫描算法的外层循环改写为 $i = 2, \dots, n$, 因为 $x_1 = b_1 / l_{11}$ 已在循环外单独给出; 同时把列扫描算法的外层循环改为 $j = 1, \dots, n - 1$, 最后再单独计算 x_n , 这样与算法逻辑是自洽的。

以上两个算法的内层循环都在不断修正右端项 b_i , 不同之处只在于循环变量 i, j 的次序不同。对它们作向量与并行分析可知:

1. 行扫描法的向量计算内核包含一步向量乘法和一个扇入求和, 而列扫描法则归结为计算若干向量的数乘与求和。
2. 二者的并行度分别大致为 $1, 2, \dots, n - 1$ 与 $n - 1, n - 2, \dots, 1$, 平均并行度都约为 $O(n/2)$ 。
3. 若有 $p \geq n - 1$ 台处理机可供利用, 则二者都可以在 $n - 1$ 个并行步内完成。

对于具有内积硬指令的向量机, 行扫描法通常更有优势; 但若机器没有专门的内积指令, 例如 CRAY-1 这一类向量机, 则内积中的扇入求和部分往往比较费时, 相比之下列扫描法由于其向量相加内核更容易实现链接, 因而可能更为有利。

进一步看数据存储方式。行扫描法要求矩阵以行方式存储, 而列扫描法则要求矩阵以列方式存储。在实际应用中, 初始问题给出的数据组织方式往往会影响扫描算法的选取。另外, 从数据调度的角度看, 内积计算通常从内存中读取两个向量后生成一个标量, 而向量相加运算则需要频繁读取两个向量并产生第三个向量写回内存; 只有在能充分使用与复用向量寄存器时, 这一差别才不至于造成明显影响。

7.1.2 列扫描法的并行实现

下面讨论列扫描法在局部存储系统与共享存储系统上的并行实现。至于行扫描法, 只需把相应的存储方式改为按行交错存储, 分析方法是类似的。

对于局部存储并行系统, 可把 L 与 b 按列交错地分配到各处理机上。若共有 p 台处理机, 则可令处理机 $P_1, P_2, \dots, P_p$ 分别存放

$$\{b_1, b_{p+1}, \dots\}, \{b_2, b_{p+2}, \dots\}, \dots, \{b_p, b_{2p}, \dots\},$$

以及与之对应的矩阵列

$$\{L_1, L_{p+1}, \dots\}, \{L_2, L_{p+2}, \dots\}, \dots, \{L_p, L_{2p}, \dots\}.$$

这里 L_i 表示矩阵 L 的第 i 列。

在这种分配下, 首先由含 b_1 和 L_1 的处理机 P_1 求出 x_1 并广播到所有处理机, 每台处理机随后修正自己所拥有的右端分量

$$b_i \leftarrow b_i - l_{i1} x_1, \quad i = 2, 3, \dots, n.$$

然后由含 b_2 和 L_2 的处理机 P_2 求出 x_2 , 再广播给其他处理机; 重复这一过程即可依次得到 $x_3, x_4, \dots, x_n$.

这种实现的优点是最初若干步中各处理机关于 b_i 的修正工作大体是负载均衡的. 但随着三角系统规模逐渐缩小, 可利用的并行度不断降低, 计算后期会有越来越多的处理机处于空闲状态.

此外, 上述算法还有一个不利之处: 每一步中只有一台处理机负责计算并广播 x_j , 此时其他处理机必须等待. 为了掩盖这部分时间开销, 可以采用预先计算策略. 例如在所有处理机对右端项 b_i 做第一次修正时, 让处理机 P_2 优先修正 b_2 并随即计算、传播 x_2 , 然后再去修正其余归它管理的分量. 这样一来, 一旦各处理机完成第一次修正, x_2 已经可用, 无需再等待即可进入下一步.

对于共享存储并行系统, 可采用任务的静态分配或动态分配. 例如仍可使用上面的按列交错存储方案, 指定处理机 P_i 负责列 $i, i+p, i+2p, \dots$ 的计算工作. 这与分布式系统中的预先计算列扫描法很相似, 不同之处只是需要设置一个同步机制, 保证在第 j 步开始前 x_{j-1} 已经可知.

7.1.3 块列扫描法

当 $n \gg p$ 时, 仅做逐列扫描往往不足以充分利用机器. 这时应考虑列扫描的分块形式. 假设 $p \mid n$, 并记

$$k = \frac{n}{p}.$$

令

$$L^{(0)} = L, \quad b^{(0)} = b,$$

并设对每个 $j = 0, 1, \dots, k-1$, 矩阵或向量 $L^{(j)}, x^{(j)}, b^{(j)}$ 的阶分别为 $n - jp$, 其分块形式为

$$L^{(j)} = \begin{bmatrix} L_{11}^{(j)} & 0 \\ L_{21}^{(j)} & L_{22}^{(j)} \end{bmatrix}, \quad x^{(j)} = \begin{bmatrix} x_1^{(j)} \\ x_2^{(j)} \end{bmatrix}, \quad b^{(j)} = \begin{bmatrix} b_1^{(j)} \\ b_2^{(j)} \end{bmatrix},$$

其中 $L_{11}^{(j)}$ 的阶为 p , 且 $x_1^{(j)}, b_1^{(j)}$ 的维数均为 p .

于是块列扫描算法可写为:

算法 3.3 块列扫描算法

```

for  $j = 0$  to  $k - 2$  do
    用列扫描法求解  $L_{11}^{(j)} x_1^{(j)} = b_1^{(j)}$ 
     $b_2^{(j+1)} \leftarrow b_2^{(j)} - L_{21}^{(j)} x_1^{(j)}$ 
end for
    用列扫描法求解  $L^{(k-1)} x^{(k-1)} = b^{(k-1)}$ 

```

这个算法把一个 n 维三角方程组的求解化为若干个 p 维三角方程组的求解以及若干次矩阵-向量乘法, 因而在 $n \gg p$ 时通常更适合并行机实现.

7.2 一般线性方程组的并行求解

对于系数矩阵稠密的线性方程组

$$Ax = b,$$

其中

$$A = (a_{ij})_{n \times n}$$

是非奇异矩阵, $b = (b_1, b_2, \dots, b_n)^T$ 与 $x = (x_1, x_2, \dots, x_n)^T$ 分别是已知向量与待求向量. 经典的直接解法包括 LU 分解法、Gauss 消去法以及列主元消去法等.

7.2.1 Gauss 消去法

求解一般形式线性方程组的最基本串行直接方法是 Gauss 消去法, 它本质上也是一种 LU 分解法. 通过一系列初等变换, 可把系数矩阵 A 分解成下三角矩阵 L 与上三角矩阵 U 的乘积:

$$PAQ = LU,$$

其中 P, Q 分别是执行矩阵行、列交换的置换矩阵. 若分解过程中不选主元, 则 $P = Q = I$. 此时原问题可归结为两个三角方程组的求解.

从增广矩阵的观点看, Gauss 消去实际上是逐次对方程组进行降阶处理. 对每个

$$k = 1, 2, \dots, n-1,$$

可作如下消元计算:

$$a_{kj}^{(k)} = \frac{a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}}, \quad j = k+1, \dots, n+1,$$

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - a_{ik}^{(k-1)} a_{kj}^{(k)}, \quad i = k+1, \dots, n, \quad j = k+1, \dots, n+1.$$

最后在 $k = n$ 时, 有

$$a_{n,n+1}^{(n)} = \frac{a_{n,n+1}^{(n-1)}}{a_{nn}^{(n-1)}}.$$

这样就把原方程组化为单位上三角方程组

$$Ux = b',$$

其中 U 的严格上三角部分由 $(a_{ij}^{(i)})_{i < j}$ 给出, 新右端项

$$b' = (a_{1,n+1}^{(1)}, a_{2,n+1}^{(2)}, \dots, a_{n,n+1}^{(n)})^T.$$

注意, Gauss 消去过程本身已经包含了向前代换的计算; 易知串行消去计算时间复杂性约为

$$O\left(\frac{2n^3}{3}\right).$$

上述消去公式具有较高的并行性. 若有 $n(n-1)$ 台处理机可供使用, 则第 k 步可先用最高并行度完成一次除法, 再用 i, j 双循环完成更新, 从而可以在大约 $3(n-1)$ 个时间步内完成全部消去. 但在实际中更有意义的是按行或按列的并行处理方式. 注意到更新公式关于下标 i, j 具有一定对称性, 不妨先取按列方式的并行计算:

算法 3.4 列 Gauss 消去法

```

for  $k = 1$  to  $n - 1$  do
  for  $j = k + 1$  to  $n + 1$  in parallel do
     $a_{kj} \leftarrow a_{kj} / a_{kk}$ 
  end for
  for  $j = k + 1$  to  $n + 1$  do

```

```

    for  $i = k + 1$  to  $n$  in parallel do
         $a_{ij} \leftarrow a_{ij} - a_{kj}a_{ik}$ 
    end for
end for
 $a_{n,n+1} \leftarrow a_{n,n+1}/a_{nn}$ 

```

若设共有 n 台处理机, 则在第 k 步中, 先做一次并行度为 $n - k + 1$ 的并行除法, 再做 $n - k + 1$ 次并行度为 $n - k$ 的并行乘、减计算. 因而消去过程的并行步数满足

$$\sum_{k=1}^{n-1} [1 + 2(n - k + 1)] + 1 = O(n^2).$$

再利用列扫描算法并行实现回代过程, 其步数约为 $2(n - 1)$, 从而并行计算时间由串行算法的 $O(n^3)$ 降低了一个数量级.

7.2.2 列主元消去法

以上算法是在假定 A 的各级主子式均不为零的前提下得到的, 否则 Gauss 消去法中的除法便无法进行. 即使各级主子式都不为零, 若其中某些主元绝对值很小, 也可能导致计算不稳定, 从而影响最终解的精度. 因此, 在实际计算中通常需要在消元过程中引入主元选择.

最常用的是局部选主元策略. 对于算法 3.4 的第 k 步, 在当前列中寻找按模最大的元素

$$|a_{i_k k}^{(k-1)}| = \max_{i=k, \dots, n} |a_{ik}^{(k-1)}|.$$

若 $i_k \neq k$, 则先把增广矩阵 $(A | b)$ 的第 i_k 行与第 k 行交换, 再继续做第 k 级消元. 这里把 slide 中容易误写的 $i_k > k$ 改正为 $i_k \geq k$, 因为主元本来就可能位于第 k 行.

这一选主元过程本质上是对当前列元素逐次比较以找出最大模元素, 因而可以采用扇入比较并行实现. 于是, 主元搜索本身的总并行步数为

$$\sum_{k=1}^{n-1} \log_2(n - k + 1) = \sum_{k=2}^n \log_2 k.$$

与不选主元的情形相比, 列主元消去法增加的主要代价就在于逐列搜索与必要的行交换, 但它通常能显著改善算法的数值稳定性.

7.2.3 LU 分解的向量形式

从算术运算的角度看, LU 分解的基本步骤无非是形如

$$a_{ij} \leftarrow a_{ij} - l_{ik}a_{kj}$$

的三重循环. 因而与矩阵乘法类似, 只要考虑循环指标 i, j, k 的不同次序, 就有 6 种基本形式; 若再结合按行存储与按列存储两种数据组织方式, 共可得到 12 种类型.

下面引进记号

$$\begin{aligned} (a_i)_k &= (a_{i1}, a_{i2}, \dots, a_{ik}), & (a^j)_k &= (a_{1j}, a_{2j}, \dots, a_{kj})^T, \\ (a_i)^k &= (a_{ik}, a_{i,k+1}, \dots, a_{in}), & (a^j)^k &= (a_{kj}, a_{k+1,j}, \dots, a_{nj})^T. \end{aligned}$$

一般认为, 直接更新型算法在合适的数据存储与通信环境下较有希望, 而内积型算法由于数据分布不理想、通信开销较高, 往往不太适合大规模并行系统. 下面给出 6 种基本向量算法.

算法 3.5 LU 分解的行向直接更新算法

```

for  $k = 1$  to  $n - 1$  do
  for  $i = k + 1$  to  $n$  do
     $l_{ik} \leftarrow a_{ik} / a_{kk}$ 
     $(a_i)^{(k+1)} \leftarrow (a_i)^{(k+1)} - l_{ik} (a_k)^{(k+1)}$ 
  end for
end for

```

算法 3.6 LU 分解的列向直接更新算法

```

for  $k = 1$  to  $n - 1$  do
   $(l^k)^{(k+1)} \leftarrow a_{kk}^{-1} (a^k)^{(k+1)}$ 
  for  $j = k + 1$  to  $n$  do
     $(a^j)^{(k+1)} \leftarrow (a^j)^{(k+1)} - a_{kj} (l^k)^{(k+1)}$ 
  end for
end for

```

算法 3.7 LU 分解的行向延迟更新算法

```

for  $i = 2$  to  $n$  do
  for  $k = 1$  to  $i - 1$  do
     $l_{ik} \leftarrow a_{ik} / a_{kk}$ 
     $(a_i)^{(k+1)} \leftarrow (a_i)^{(k+1)} - l_{ik} (a_k)^{(k+1)}$ 
  end for
end for

```

算法 3.8 LU 分解的列向延迟更新算法

```

for  $j = 2$  to  $n$  do
   $(l^{j-1})^j \leftarrow a_{j-1,j-1}^{-1} (a^{j-1})^j$ 
  for  $k = 1$  to  $j - 1$  do
     $(a^j)^{(k+1)} \leftarrow (a^j)^{(k+1)} - a_{kj} (l^k)^{(k+1)}$ 
  end for
end for

```

算法 3.9 LU 分解的行向内积算法

```

for  $i = 2$  to  $n$  do
  for  $j = 1$  to  $i - 1$  do
     $l_{ij} \leftarrow (a_{ij} - (l_i)_{j-1} (a^j)_{j-1}) / a_{jj}$ 
  end for

```

```

for  $j = i$  to  $n$  do
     $a_{ij} \leftarrow a_{ij} - (l_i)_{i-1} (a^j)_{i-1}$ 
end for
end for

```

算法 3.10 LU 分解的列向内积算法

```

for  $j = 2$  to  $n$  do
    for  $i = j$  to  $n$  do
         $l_{i,j-1} \leftarrow (a_{i,j-1} - (l_i)_{j-2} (a^{j-1})_{j-2}) / a_{j-1,j-1}$ 
    end for
    for  $i = 2$  to  $j$  do
         $a_{ij} \leftarrow a_{ij} - (l_i)_{i-1} (a^j)_{i-1}$ 
    end for
end for

```

需要说明的是, 上面几个算法中有些 slide 的下标写法较为拥挤, 这里已统一改写成了便于阅读的形式. 例如算法 3.10 中把分母明确写成 $a_{j-1,j-1}$, 并把 $l_{i,j-1}$ 与 $a_{i,j-1}$ 的下标分开书写, 以免与幂指数混淆.

概括地说, 这 6 种形式的最内层基本向量运算决定了它们的性质:

1. 算法 3.5 与算法 3.6 的基本向量运算都是链三元组, 只要数据齐备, 前者更新各行, 后者更新各列, 因而称为直接更新算法.
2. 算法 3.7 与算法 3.8 的基本向量运算也是链三元组, 但它们是被反复应用并逐步累加形成一个线性组合, 因而称为延迟更新算法.
3. 算法 3.9 与算法 3.10 则以内积为主要计算内核, 故称内积算法.

对算法 3.5 作一个简单分析可知, 其主要计算是对当前主行之下若干行所做的链三元组更新. 若机器要求向量由顺序可寻址元素组成, 则矩阵 A 最好按行存储. 在第 k 级, 要对第 $k+1$ 至第 n 行分别减去第 k 行的若干倍, 共进行 $n-k$ 次向量链三元组运算, 且向量长度都等于 $n-k$. 因而关于更新 A 的全部运算的平均向量长度为

$$\frac{\sum_{k=1}^{n-1} (n-k)^2}{\sum_{k=1}^{n-1} (n-k)} = O\left(\frac{2n}{3}\right).$$

从存储与通信的角度看, 按行存储的 kij 型较适合行向直接更新, 按列存储的 kji 型较适合列向直接更新; 而像 ijk 、 jik 一类内积形式, 无论采用按行还是按列存储, 一般都存在较大的通信或负载不均衡问题, 不太适用于大规模并行环境.

7.2.4 LU 分解的加边算法

为了递推地构造 LU 分解, 记

$$A_j = \begin{bmatrix} a_{11} & \cdots & a_{1j} \\ \vdots & \ddots & \vdots \\ a_{j1} & \cdots & a_{jj} \end{bmatrix}, \quad j = 1, 2, \dots, n,$$

即 A 的第 j 个主子矩阵. 它可以看作在 A_{j-1} 的右边和下边各加一条边所形成的分块矩阵:

$$A_j = \begin{bmatrix} A_{j-1} & (a^j)_{j-1} \\ (a_j)_{j-1} & a_{jj} \end{bmatrix}.$$

若已知

$$A_{j-1} = L_{j-1}U_{j-1},$$

则 A_j 的分解可写成

$$A_j = L_j U_j = \begin{bmatrix} L_{j-1} & 0 \\ (l_j)_{j-1} & 1 \end{bmatrix} \begin{bmatrix} U_{j-1} & (u^j)_{j-1} \\ 0 & u_{jj} \end{bmatrix}.$$

于是, 待定量是 L_j 的第 j 行前 $j-1$ 个元素 $(l_j)_{j-1}$, 以及 U_j 的第 j 列前 $j-1$ 个元素 $(u^j)_{j-1}$ 和对角元 u_{jj} . 它们可由两个三角方程组依次求出:

$$(l_j)_{j-1} U_{j-1} = (a_j)_{j-1}, \quad U_{j-1}^T (l_j)_{j-1}^T = (a_j)_{j-1}^T. \quad (3.2.7)$$

$$\begin{bmatrix} L_{j-1} & 0 \\ (l_j)_{j-1} & 1 \end{bmatrix} \begin{bmatrix} (u^j)_{j-1} \\ u_{jj} \end{bmatrix} = \begin{bmatrix} (a^j)_{j-1} \\ a_{jj} \end{bmatrix}. \quad (3.2.8)$$

因此, LU 分解的加边算法可表述如下:

算法 3.11 LU 分解的加边算法

```

for  $j = 2$  to  $n$  do
    求解方程组 (3.2.7)
    求解方程组 (3.2.8)
end for

```

这里最关键的计算工作就是三角方程组的求解, 因而该算法天然适合与已有的三角系统并行算法相配合.

7.2.5 Dongarra-Eisenstat 算法

下面介绍 Dongarra-Eisenstat 算法. 它的基本运算是矩阵-向量乘法, 因而很适合向量机与并行机实现. 为了写出它的递推关系, 把 A, L, U 在第 j 级作如下分块:

$$\begin{bmatrix} A_{j-1} & (a^j)_{j-1} & A_{j-1}^{(1)} \\ (a_j)_{j-1} & a_{jj} & (a_j)^{j+1} \\ A_{j-1}^{(2)} & (a^j)^{j+1} & A_{j-1}^{(3)} \end{bmatrix} = \begin{bmatrix} L_{j-1} & 0 & 0 \\ (l_j)_{j-1} & 1 & 0 \\ L_{j-1}^{(2)} & (l^j)^{j+1} & I \end{bmatrix} \begin{bmatrix} U_{j-1} & (u^j)_{j-1} & U_{j-1}^{(1)} \\ 0 & u_{jj} & (u_j)^{j+1} \\ 0 & 0 & U_{j-1}^{(3)} \end{bmatrix}. \quad (3.2.9)$$

设已完成分解 $A_{j-1} = L_{j-1}U_{j-1}$, 并已求得 $(l_j)_{j-1}$ 、 $L_{j-1}^{(2)}$ 、 $(u^j)_{j-1}$ 与 $U_{j-1}^{(1)}$. 在第 j 级, 需要进一步计算 U 的第 j 行与 L 的第 j 列. 由式 (3.2.9) 可得

$$\begin{cases} u_{jj} = a_{jj} - (l_j)_{j-1}(u^j)_{j-1}, \\ (u_j)^{j+1} = (a_j)^{j+1} - (l_j)_{j-1}U_{j-1}^{(1)}, \\ (l^j)^{j+1} = [(a^j)^{j+1} - L_{j-1}^{(2)}(u^j)_{j-1}]/u_{jj}. \end{cases} \quad (3.2.10)$$

从这个公式可以看出, 算法的核心正是矩阵-向量乘法. 它自然导出两种实现形式:

算法 3.12 LU 分解的 Dongarra-Eisenstat 算法**(1) 线性组合形式**

```

for  $j = 1$  to  $n$  do
    for  $k = 1$  to  $j - 1$  do
         $(a_j)^j \leftarrow (a_j)^j - l_{jk}(a_k)^j$ 
    end for
    for  $k = 1$  to  $j - 1$  do
         $(a^j)^{(j+1)} \leftarrow (a^j)^{(j+1)} - a_{kj}(l^k)^{(j+1)}$ 
    end for
     $(l^j)^{(j+1)} \leftarrow a_{jj}^{-1}(a^j)^{(j+1)}$ 
end for

```

(2) 内积形式

```

for  $j = 1$  to  $n$  do
    for  $i = j$  to  $n$  do
         $a_{ji} \leftarrow a_{ji} - (l_j)_{j-1}(a^i)_{j-1}$ 
    end for
    for  $i = j + 1$  to  $n$  do
         $a_{ij} \leftarrow a_{ij} - (l_i)_{j-1}(a^j)_{j-1}$ 
    end for
     $(l^j)^{(j+1)} \leftarrow a_{jj}^{-1}(a^j)^{(j+1)}$ 
end for

```

容易看出, 算法 3.12 的两种形式中, 第一步都在原地计算 u_{jj} 与 $(u_j)^{j+1}$, 计算结果存放在 $a_{jj}, a_{j,j+1}, \dots, a_{jn}$ 这些存储单元中; 第二、三步则用来求出 $(l^j)^{j+1}$.

7.2.6 LU 分解矩阵与多重右端项

有时我们不仅要利用消去过程求解单个右端项, 还需要显式写出完整的 LU 分解所对应的两个三角矩阵. 在不选主元的情形下, 第 k 步消元可看作对第 $k - 1$ 次修正矩阵左乘一个初等变换矩阵 M_k , 因而有

$$M_{n-1}M_{n-2}\cdots M_1A = U.$$

其中

$$M_k = \begin{bmatrix} I_{k-1} & 0 & 0 \\ 0 & \frac{1}{a_{kk}^{(k-1)}} & 0 \\ 0 & -\frac{(a^k)^{k+1}}{a_{kk}^{(k-1)}} & I_{n-k} \end{bmatrix}, \quad M_k^{-1} = \begin{bmatrix} I_{k-1} & 0 & 0 \\ 0 & a_{kk}^{(k-1)} & 0 \\ 0 & (a^k)^{k+1} & I_{n-k} \end{bmatrix} \equiv L_k.$$

于是

$$L = L_1L_2\cdots L_{n-1} = (a_{ij}^{(j-1)})_{i \geq j}.$$

这说明在算法 3.4 的覆盖存储方式下, 分解结束后矩阵 A 的严格上三角部分对应于 U 的严格上三角部分, 而下三角部分恰好给出 L .

若 A 的 LU 分解已经求出, 而需要求解多重右端项方程组

$$AX = B,$$

其中 X, B 分别为 $n \times q$ 阶的解矩阵与右端项矩阵, 则它等价于两个三角矩阵方程

$$LY = B, \quad UX = Y.$$

如果 LU 分解尚未事先完成, 则可直接对增广矩阵 $(A | B)$ 执行 Gauss 消去. 此时算法 3.4 中列指标的取值范围由 $k+1, \dots, n+1$ 扩展为 $k+1, \dots, n+q$. 消去结束后, 只需再做一次多重右端项单位上三角方程组的回代即可. 从并行度的角度看, 这时按行方式组织消元通常更为有利.

7.3 三对角线性方程组的并行解法

许多基本数值计算问题的求解都要归结为三对角线性方程组. 例如, 三次样条插值函数的计算、常微分方程两点边值问题的有限差分离散、偏微分方程隐式差分格式等, 都会产生这类系统. 因而三对角方程组的并行与向量算法一直是数值并行计算中的重要课题.

设所要求解的三对角线性方程组为

$$Ax = r,$$

其中

$$A = \begin{bmatrix} a_1 & b_1 & & & \\ c_2 & a_2 & b_2 & & \\ & \ddots & \ddots & \ddots & \\ & & c_{n-1} & a_{n-1} & b_{n-1} \\ & & & c_n & a_n \end{bmatrix}, \quad x = (x_1, x_2, \dots, x_n)^T, \quad r = (r_1, r_2, \dots, r_n)^T.$$

若记

$$A = LU,$$

其中

$$L = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & \ddots & \ddots & \ddots & \\ & & l_n & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_1 & b_1 & & & \\ & u_2 & b_2 & & \\ & & \ddots & \ddots & \\ & & & u_n & \end{bmatrix},$$

则由 $A = LU$ 可得递推关系

$$\begin{cases} u_1 = a_1, \\ l_i = c_i / u_{i-1}, & i = 2, 3, \dots, n, \\ u_i = a_i - l_i b_{i-1}, & i = 2, 3, \dots, n. \end{cases} \quad (3.3.5)$$

令 $y = Ux$, 则先解方程组 $Ly = r$:

$$\begin{cases} y_1 = r_1, \\ y_i = r_i - l_i y_{i-1}, & i = 2, \dots, n, \end{cases} \quad (3.3.6)$$

再解方程组 $Ux = y$:

$$\begin{cases} x_n = y_n/u_n, \\ x_i = (y_i - b_i x_{i+1})/u_i, \quad i = n-1, \dots, 1. \end{cases} \quad (3.3.7)$$

7.3.1 Stone 算法

式 (3.3.5)、(3.3.6) 与 (3.3.7) 都是递推计算问题, 其中 (3.3.6) 与 (3.3.7) 是线性递推, 而 (3.3.5) 是非线性递推. 为了把它转化成适合并行计算的线性递推, Stone 引入序列

$$\begin{cases} g_0 = 1, \\ g_1 = a_1, \\ g_i = a_i g_{i-1} - c_i b_{i-1} g_{i-2}, \quad i = 2, \dots, n. \end{cases} \quad (3.3.8)$$

由此可知

$$u_i = \frac{g_i}{g_{i-1}}, \quad l_i = \frac{c_i g_{i-2}}{g_{i-1}}.$$

把二阶递推改写成一阶矩阵递推, 令

$$Q_i = \begin{bmatrix} g_i \\ g_{i-1} \end{bmatrix},$$

则有

$$Q_i = \begin{bmatrix} a_i & -c_i b_{i-1} \\ 1 & 0 \end{bmatrix} \begin{bmatrix} g_{i-1} \\ g_{i-2} \end{bmatrix} \triangleq G_i Q_{i-1} = \left(\prod_{j=2}^i G_j \right) Q_1. \quad (3.3.10)$$

同样地, 对线性递推 (3.3.6) 也可作类似表示. 例如令

$$Y_i = \begin{bmatrix} y_i \\ 1 \end{bmatrix},$$

则

$$Y_i = \begin{bmatrix} -l_i & r_i \\ 0 & 1 \end{bmatrix} \begin{bmatrix} y_{i-1} \\ 1 \end{bmatrix} \triangleq H_i Y_{i-1} = \left(\prod_{j=2}^i H_j \right) Y_1. \quad (3.3.11)$$

于是, Q_n 与 Y_n 都可以借助熟知的递推倍增法并行计算. 若 $n = 2^k$, 则这两个量都可在 $k = \log_2 n$ 步内求得. Stone 算法的总运算量为

$$O(n \log_2 n).$$

从标量运算量看, 它不如串行 Thomas 算法那样高效; 但若以向量运算步数作为主要指标, 则它只需要 $O(\log_2 n)$ 个向量步骤, 因而仍然是很有吸引力的加速方法.

7.3.2 循环约化法

另一种非常适合并行计算并且在向量机上取得成功的方法是循环约化法 (Cyclic Reduction Method), 它也被称为奇偶约化法. 其基本思想是在偶数编号的方程中消去奇数编号的未知量, 从而得到一个规模减半的新的三对角方程组.

为说明其思想, 可先设 $n = 8$. 通过并行地对偶数编号方程做初等变换, 可以得到只含偶数未知量 x_2, x_4, x_6, x_8 的新三对角系统. 一旦这个新系统解出, 再把它们代回奇数编号的诸方程中, 便

可直接求得 x_1, x_3, x_5, x_7 . 这一过程还可以继续递归地应用到规模进一步减半的新系统上.

一般地, 每一步都把规模为 2^k 的方程组约化为规模为 2^{k-1} 的方程组. 在第 k 步之后, 只剩下一个只含单个未知数的方程, 求出该未知数后即可回代恢复全部解. 令 $n = 2^k$ 只是为了方便叙述; 对任意 n , 整个过程都可以作类似处理.

下面给出循环约化法的一种矩阵化描述. 先把奇数编号变量排在前面, 记 $A = A^{(0)}$, 并令 P_1 为相应的置换矩阵, 则

$$A^{(1)} = P_1 A^{(0)} P_1^T = \begin{bmatrix} D_1^{(1)} & F^{(1)} \\ G^{(1)} & D_2^{(1)} \end{bmatrix}, \quad (3.3.15)$$

其中

$$D_1^{(1)} = \text{diag}(a_1, a_3, \dots, a_{2^k-1}), \quad D_2^{(1)} = \text{diag}(a_2, a_4, \dots, a_{2^k}),$$

$F^{(1)}$ 是下双对角矩阵, 其主对角线上是奇数编号的 b , 次对角线上是奇数编号的 c ; $G^{(1)}$ 是上双对角矩阵, 其主对角线上是偶数编号的 c , 次对角线上是偶数编号的 b . 这里约定 $c_1 = b_{2^k} = 0$.

对矩阵 $A^{(1)}$ 作块分解

$$A^{(1)} = L_1 T_1 U_1,$$

其中 L_1 、 U_1 分别为单位块下三角矩阵与单位块上三角矩阵, T_1 为块对角矩阵. 具体有

$$\begin{aligned} L_1 &= \begin{bmatrix} I & 0 \\ G^{(1)}(D_1^{(1)})^{-1} & I \end{bmatrix}, \quad U_1 = \begin{bmatrix} I & (D_1^{(1)})^{-1}F^{(1)} \\ 0 & I \end{bmatrix}, \\ T_1 &= \begin{bmatrix} D_1^{(1)} & 0 \\ 0 & D_2^{(1)} - G^{(1)}(D_1^{(1)})^{-1}F^{(1)} \end{bmatrix} = \begin{bmatrix} T_{11} & 0 \\ 0 & T_{22} \end{bmatrix}. \end{aligned} \quad (3.3.16)$$

等价地,

$$L_1^{-1} P_1 A^{(0)} P_1^T U_1^{-1} = T_1. \quad (3.3.17)$$

容易看出, T_{22} 正是第一次约化后得到的较小规模三对角方程组的系数矩阵. 因而可以对 T_{22} 重复应用同样的过程. 经过 k 步之后, 可得到分解

$$A = (P_1^T L_1 P_2^T L_2 \cdots P_k^T L_k) T_k (U_k P_k U_{k-1} P_{k-1} \cdots U_1 P_1) \equiv Q T_k E, \quad (3.3.18)$$

其中 T_k 是对角矩阵.

注意此时 Q 与 E 虽然不再分别是下三角阵与上三角阵, 但仍可以用来求解原方程组 $Ax = r$. 令

$$T_k E x = y,$$

先解

$$Q y = r.$$

若记 $z_0 = r$, 则可通过递推

$$L_i z_i = P_i z_{i-1}, \quad i = 1, \dots, k, \quad (3.3.19)$$

得到 $y = z_k$. 然后再解

$$E x = T_k^{-1} y.$$

循环约化法可以看作是对某个置换后的矩阵方程 PAP^T 实施 Gauss 消去. 因此, 在不选主元的 Gauss 消去本身稳定的条件下, 循环约化法同样是数值稳定的. 例如, 当 A 为对称正定矩阵或

具有主对角占优性质时, 循环约化法就是稳定的.

另外, 循环约化法还是相容算法. 在整个计算过程中, 平均向量长度为

$$\frac{1}{k} \sum_{i=1}^k 2^{k-i} = \frac{n-1}{k}. \quad (3.3.20)$$

历史上的向量机实验表明, 当问题规模足够大时, 循环约化法通常能获得明显的向量加速; 同时它还可以自然推广到块三对角方程组的求解.

7.3.3 分裂法

下面介绍 H.H. Wang 提出的求解三对角方程组的并行算法, 即分裂法 (Partition Method). 分裂法是贯彻分而治之原则的一个成功例子, 曾受到广泛重视. 它的基本思想是先把原方程组分裂为若干个子方程组, 然后在此基础上对各子方程组分别施加初等变换, 最后把整个系数矩阵化为对角形.

在正式叙述一般算法之前, 先以 $n = 16$ 的简单例子说明其消元过程. 第一步先把矩阵分裂成 4×4 的块三对角形式. 这时原三对角矩阵可写成

$$\left[\begin{array}{ccc|ccc|ccc|ccc} a_1 & b_1 & & & & & & & & & & & & & & \\ c_2 & a_2 & b_2 & & & & & & & & & & & & & \\ & c_3 & a_3 & b_3 & & & & & & & & & & & & \\ & & c_4 & a_4 & b_4 & & & & & & & & & & & \\ & & & c_5 & a_5 & b_5 & & & & & & & & & & \\ & & & & c_6 & a_6 & b_6 & & & & & & & & & \\ & & & & & c_7 & a_7 & b_7 & & & & & & & & \\ & & & & & & c_8 & a_8 & b_8 & & & & & & & \\ & & & & & & & c_9 & a_9 & b_9 & & & & & & \\ & & & & & & & & c_{10} & a_{10} & b_{10} & & & & & \\ & & & & & & & & & c_{11} & a_{11} & b_{11} & & & & \\ & & & & & & & & & & c_{12} & a_{12} & b_{12} & & & \\ & & & & & & & & & & & c_{13} & a_{13} & b_{13} & & \\ & & & & & & & & & & & & c_{14} & a_{14} & b_{14} & \\ & & & & & & & & & & & & & c_{15} & a_{15} & b_{15} \\ & & & & & & & & & & & & & & c_{16} & a_{16} \end{array} \right] \quad (3.3.21)$$

其中块分界线仅用来表示分裂后的结构, 不改变矩阵原来的三对角性质.

接着可以依次进行如下并行消元:

1. 同时对四个主对角块作块内消元, 使它们都化为上三角形.
2. 同时消去 c_2, c_6, c_{10}, c_{14} , 再同时消去 c_3, c_7, c_{11}, c_{15} 与 c_4, c_8, c_{12}, c_{16} . 这样, 除第 4, 8, 12, 16 列外, 矩阵已变为三角形. 在这一过程中产生了一批以 f 标记的新非零元素.
3. 同时消去 b_2, b_6, b_{10}, b_{14} , 再同时消去 b_1, b_5, b_9, b_{13} 与 b_4, b_8, b_{12} . 此后, 除第 4, 8, 12, 16 列外,

矩阵已化为对角形, 并进一步产生了一批以 q 标记的新非零元素.

经过前 3 步以后, 矩阵的非零结构可概括写成

$$\left[\begin{array}{cc|cc|cc|cc} a_1 & g_1 & & & & & & \\ & a_2 & g_2 & & & & & \\ & & a_3 & b_3 & & & & \\ & & & a_4 & & & & \\ \hline & c_5 & a_5 & g_5 & & & & \\ & f_6 & & a_6 & g_6 & & & \\ & f_7 & & & a_7 & b_7 & & \\ & f_8 & & & & a_8 & & \\ \hline & & & c_9 & a_9 & g_9 & & \\ & & & f_{10} & & a_{10} & g_{10} & \\ & & & f_{11} & & & a_{11} & b_{11} \\ & & & f_{12} & & & & a_{12} \\ \hline & & & & & c_{13} & a_{13} & g_{13} \\ & & & & & f_{14} & & a_{14} & g_{14} \\ & & & & & f_{15} & & & a_{15} & b_{15} \\ & & & & & f_{16} & & & & a_{16} \end{array} \right] \quad (3.3.22)$$

4. 继续对主对角线下方第 ik 列元素做消元 ($i = 1, 2, \dots, p-1$), 矩阵化为三角形.
5. 再消去主对角线上方第 ik 列元素 ($i = 1, 2, \dots, p-1$), 最终把矩阵化为对角形.
6. 最后逐分量求解

$$x_i = \frac{r_i}{a_i}, \quad i = 1, \dots, n.$$

若把 slide 中被反光遮挡的若干乘号和下标按上下文整理, 则第 2、3 步可写成下列并行伪代码.

Step (2): 主对角块的前向消元

```

for  $i = 1$  to  $p - 1$  in parallel do
     $f_{ik+1} \leftarrow c_{ik+1}$ 
end for
for  $i = 0$  to  $p - 1$  do
    for  $j = 2$  to  $k$  in parallel do
         $c_{ik+j} \leftarrow c_{ik+j} / a_{ik+j-1}$ 
         $a_{ik+j} \leftarrow a_{ik+j} - c_{ik+j} b_{ik+j-1}$ 
         $r_{ik+j} \leftarrow r_{ik+j} - c_{ik+j} r_{ik+j-1}$ 
    end for
end for
for  $i = 1$  to  $p - 1$  do
    for  $j = 2$  to  $k$  in parallel do
         $f_{ik+j} \leftarrow -c_{ik+j} f_{ik+j-1}$ 
    end for
end for

```

Step (3): 主对角块的后向消元

```

for  $i = 1$  to  $p - 1$  in parallel do
     $g_{ik-1} \leftarrow b_{ik-1}$ 
end for
for  $i = 0$  to  $p - 1$  do
    for  $j = 2$  to  $k$  in parallel do
         $b_{ik+j-1} \leftarrow b_{ik+j-1} / a_{ik+j}$ 
         $g_{ik+j-1} \leftarrow -b_{ik+j-1} g_{ik+j}$ 
         $r_{ik+j-1} \leftarrow r_{ik+j-1} - b_{ik+j-1} r_{ik+j}$ 
         $f_{ik+j-1} \leftarrow f_{ik+j-1} - b_{ik+j-1} f_{ik+j}$ 
    end for
end for
for  $i = 1$  to  $p - 1$  in parallel do
     $b_{ik} \leftarrow b_{ik} / a_{ik+1}$ 
     $a_{ik} \leftarrow a_{ik} - b_{ik} f_{ik+1}$ 
     $g_{ik} \leftarrow -b_{ik} g_{ik+1}$ 
     $r_{ik} \leftarrow r_{ik} - b_{ik} r_{ik+1}$ 

```

end for

当 $k = 2$ 时, 上述第 2、3 步中的某些块内递推会退化为一步计算, 因而可作相应简化. 第 4、5 步本质上分别是对保留下来的边界列做一次下消元和一次上消元; 它们不再产生新的非零元, 因而结构上比前两步更简单.

分裂法是一种相容的并行算法. 它可以看作是不选主元 Gauss 消去法在分而治之思想下的成功应用与发展. 当矩阵 A 具有主对角占优性质时, 该方法是稳定的; 并且它同时适用于并行机和向量机.

7.3.4 分段追赶并行算法

在串行计算机上求解三对角方程组, 最常用而有效的方法是追赶法. 分段追赶并行算法的出发点, 就是把熟知的串行追赶法加以分段化, 再由此导出适合并行实现的求解公式. 由于各段追赶在适当意义下是彼此独立的, 因而这种思路天然适合并行计算.

先写出求解三对角方程组的串行追赶公式. 对于前向“追”的过程, 有

$$\begin{cases} p_j = c_j q_{j-1} + a_j, & q_0 \equiv 0, & j = 1, 2, \dots, n, \\ q_j = -\frac{b_j}{p_j}, & j = 1, 2, \dots, n-1, \\ u_j = \frac{r_j - c_j u_{j-1}}{p_j}, & u_0 \equiv 0, & j = 1, 2, \dots, n. \end{cases} \quad (3.3.23)$$

而对应的“赶”的过程为

$$\begin{cases} x_n = u_n, \\ x_j = q_j x_{j+1} + u_j, & j = n-1, \dots, 1. \end{cases} \quad (3.3.24)$$

类似地, 还可以写出一种“反向”的追赶公式:

$$\begin{cases} P_j = b_j Q_{j+1} + a_j, & Q_{n+1} \equiv 0, & j = n, n-1, \dots, 1, \\ Q_j = -\frac{c_j}{P_j}, & j = n, n-1, \dots, 2, \\ U_j = \frac{r_j - b_j U_{j+1}}{P_j}, & U_{n+1} \equiv 0, & j = n, n-1, \dots, 1. \end{cases} \quad (3.3.25)$$

其对应的“赶”的过程则为

$$\begin{cases} x_1 = U_1, \\ x_{j+1} = Q_{j+1} x_j + U_{j+1}, & j = 1, 2, \dots, n-1. \end{cases} \quad (3.3.26)$$

通常把式 (3.3.23)、(3.3.24) 称为第 1 类追赶公式, 把式 (3.3.25)、(3.3.26) 称为第 2 类追赶公式.

从推导这两类追赶公式的思路出发, 可以建立分段追赶的计算公式, 从而实现原问题的并行求解. 下面先介绍分段数 $k = 2$ 时的计算过程. 它适合于有两台处理机的并行机, 整个算法分 3 步进行:

1. 第 1 台处理机执行第 1 类追赶公式中的“追”公式 (3.3.23), 其中

$$j = 1, 2, \dots, \left\lfloor \frac{n}{2} \right\rfloor;$$

第 2 台处理机执行第 2 类追赶公式中的“追”公式 (3.3.25), 其中

$$j = n, n-1, \dots, \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

这就实现了两端同时向中间推进的并行“追”过程.

2. 在第 1 步完成后, 得到关于

$$x_{\left\lfloor \frac{n}{2} \right\rfloor}, \quad x_{\left\lfloor \frac{n}{2} \right\rfloor+1}$$

的 2×2 线性方程组

$$\begin{cases} x_{\left\lfloor \frac{n}{2} \right\rfloor} = q_{\left\lfloor \frac{n}{2} \right\rfloor} x_{\left\lfloor \frac{n}{2} \right\rfloor+1} + u_{\left\lfloor \frac{n}{2} \right\rfloor}, \\ x_{\left\lfloor \frac{n}{2} \right\rfloor+1} = Q_{\left\lfloor \frac{n}{2} \right\rfloor+1} x_{\left\lfloor \frac{n}{2} \right\rfloor} + U_{\left\lfloor \frac{n}{2} \right\rfloor+1}. \end{cases}$$

从而

$$\begin{cases} x_{\left\lfloor \frac{n}{2} \right\rfloor} = \frac{q_{\left\lfloor \frac{n}{2} \right\rfloor} U_{\left\lfloor \frac{n}{2} \right\rfloor+1} + u_{\left\lfloor \frac{n}{2} \right\rfloor}}{1 - Q_{\left\lfloor \frac{n}{2} \right\rfloor+1} q_{\left\lfloor \frac{n}{2} \right\rfloor}}, \\ x_{\left\lfloor \frac{n}{2} \right\rfloor+1} = \frac{Q_{\left\lfloor \frac{n}{2} \right\rfloor+1} u_{\left\lfloor \frac{n}{2} \right\rfloor} + U_{\left\lfloor \frac{n}{2} \right\rfloor+1}}{1 - Q_{\left\lfloor \frac{n}{2} \right\rfloor+1} q_{\left\lfloor \frac{n}{2} \right\rfloor}}. \end{cases} \quad (3.3.27)$$

因而这两个中间未知量可以同时求出.

3. 在

$$x_{\left\lfloor \frac{n}{2} \right\rfloor}, \quad x_{\left\lfloor \frac{n}{2} \right\rfloor+1}$$

已知以后, 第 1 台处理机执行第 1 类追赶公式中的“赶”公式 (3.3.24), 其中

$$j = \left\lfloor \frac{n}{2} \right\rfloor - 1, \dots, 2, 1;$$

同时第 2 台处理机执行第 2 类追赶公式中的“赶”公式 (3.3.26), 其中

$$j = \left\lfloor \frac{n}{2} \right\rfloor + 1, \left\lfloor \frac{n}{2} \right\rfloor + 2, \dots, n-1.$$

这样便得到原三对角方程组的全部解

$$x_1, x_2, \dots, x_n.$$

分段追赶并行算法的本质, 就是把原本必须串行推进的追赶过程切成若干个首尾相接的子区间, 先并行完成各区间内部的追赶, 再在分段接口处解一个小规模耦合系统, 最后并行回代. 因而它既保留了经典追赶法结构简单的优点, 又把其中最强的串行依赖压缩到了少量接口变量上.

7.4 实对称方程组的 Cholesky 分解法

关于一般情形方程组的各种并行解法, 对于对称情形仍为有效. 但注意到系数矩阵的对称性, 可以将上述算法简化, 或构造更为合适的算法.

7.4.1 对称正定方程组的 Cholesky 分解法

对于对称正定的系数矩阵 A , 其 LU 分解式变为

$$A = LL^T. \quad (7.1)$$

其中 L 为下三角阵, 称为 Cholesky 分解。于是原方程组求解化为两个联立的三角形方程组求解, 其系数矩阵互为转置, 即

$$Ly = b, \quad L^T x = y. \quad (7.2)$$

Cholesky 分解的计算格式如下。对于 $k = 1, 2, \dots, n$,

$$l_{kk} = \sqrt{a_{kk} - \sum_{i=1}^{k-1} l_{ki}^2}, \quad (7.3)$$

$$l_{kj} = \frac{a_{kj} - \sum_{i=1}^{j-1} l_{ki} l_{ji}}{l_{jj}}, \quad j = 1, 2, \dots, k-1. \quad (7.4)$$

该算法比常规的 LU 分解计算量小, 所需存储空间也更小, 稳定性好, 并且易于组织并行计算。

7.4.2 ijk 型向量算法

由于 A 对称, 只须存储 A 的下三角部分。一种实现方案可写成如下列向 Cholesky 分解格式:

算法 3.13 列向 Cholesky 分解

```

 $l_{11} \leftarrow a_{11}^{1/2}$ 
for  $j = 2$  to  $n$  do
   $(l^{j-1})^j \leftarrow l_{j-1,j-1}^{-1} (a^{j-1})^j$ 
  for  $k = 1$  to  $j-1$  do
     $(a^j)^j \leftarrow (a^j)^j - l_{jk} (l^k)^j$ 
  end for
   $l_{jj} \leftarrow a_{jj}^{1/2}$ 
end for

```

在第 j 级, 完成 L 的第 $j-1$ 列并修正 A 的第 j 列, 共包含 $j-1$ 个链三元组运算, 所用向量的长度为 $n-j+1$ 。因此, 平均向量长度为

$$\frac{(n-1) + 2(n-2) + \dots + (n-1)1}{1 + 2 + \dots + (n-1)} = O\left(\frac{n}{3}\right). \quad (3.4.3)$$

这仅是 LU 分解的一半, 这一事实使得 Cholesky 分解比 LU 分解的优越性在向量计算中不如串行计算中那样显著。实际上, 由于 Cholesky 算法利用了系数矩阵的对称性, 使串行计算工作量比 LU 分解减少近二分之一; 而在向量机上, 受启动时间的影响, Cholesky 算法的速度一般低于 LU 分解速度的 2 倍。算法 3.13 只是 Cholesky 分解的一种可能的编排方式。与 LU 分解一样, 循环指标 ijk 有 6 种排列, 因而也可给出 6 种不同编排方式的算法。

7.4.3 Cholesky 分解的并行实现

Cholesky 分解只须存储和处理 A 的下三角部分, 采用列 (或行) 交错存储对均衡各处理机负载所起的作用尤为突出。前面 Cholesky 分解的 ijk 型算法可以引出一些适合并行机和向量机的算法。

现设 A 的下三角部分按行交错存储, 处理机台数 $p \ll n$ 且 $n = mp$ 。我们考虑对应于算法

3.13 的并行方案。令

$$l_{11} = a_{11}^{1/2}, \quad l'_{i1} = a_{i1}, \quad i = 2, \dots, n. \quad (7.5)$$

则对 $j = 2, \dots, n$ 有

$$\begin{cases} l_{i,j-1} = \frac{l'_{i,j-1}}{l_{j-1,j-1}}, & j = 2, \dots, n, \\ l'_{ij} = a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}, & j = 2, \dots, n, i = j, \dots, n, \\ l_{jj} = (l'_{jj})^{1/2}, & j = 2, \dots, n. \end{cases} \quad (3.4.5)$$

这个公式的第 j 级产生 L 的第 $j-1$ 列, 算出 $l_{jj}, l'_{j+1,j}, \dots, l'_{n,j}$, 是列向分解。并行完成式 (3.4.5) 时, 通常由拥有对角元 $l_{j-1,j-1}$ 的处理机先求出该列并广播, 然后各处理机并行修正自己所拥有的行段。由于交错存储使得各处理机所承担的行数近于均匀, 故可获得较好的负载平衡。

Cholesky 分解的交错存储的 ijk 型也有 12 种情形。用 $kij(r)$ 与 $kij(c)$ 分别表示按行与列交错存储的 kij 型。这 12 种型中, 在并行系统上最有希望的是 $kji(c)$ 和 $jki(r)$ 型; 在并行-向量系统上只有 $kji(c)$ 容许长向量, 但累加向量数据有延迟损耗。

7.4.4 QR 分解简介

线性方程组 $Ax = b$ 的直接法实质上是对系数矩阵 A 作某种特殊形式的分解。除 LU 分解外, 传统的还有 QR 分解, 也有一些新型分解出现。矩阵 $A = (a_{ij})_{n \times n}$ 的 QR 分解形式为

$$A = QR, \quad (7.6)$$

其中 Q 是一个正交矩阵, R 是一个上三角矩阵。因此, 一旦实现分解, 求解 $Ax = b$ 变等价于求解上三角方程组

$$Rx = Q^T b. \quad (7.7)$$

实现分解的两种常见的途径是: Givens 变换与 Householder 变换。在串行计算机上, LU 分解运算次数为 $O(2n^3/3)$; QR 分解的运算次数, 采用 Givens 变换为 $O(2n^3)$, 约等于 LU 分解的 3 倍; 采用 Householder 变换为 $O(4n^3/3)$, 约等于 LU 分解的两倍。

这两种 QR 分解途径的基本思想都是: 从 A 出发依次用正交矩阵 (称为正交变换) 进行约化, 产生上三角矩阵 R , 而 Q 由依次所用的正交矩阵确定, 故统称正交约化方法。其显著性质是能保证数值稳定性, 而无须像选主元策略那样进行某些行或列的交换。然而, QR 分解却广泛地用于特征值计算、最小二乘问题、向量的正交化和其它场合。在这些应用中, A 常是长方形矩阵。为了简便, 我们仅限于考虑 $n \times n$ 矩阵。

7.4.5 Givens 约化法

一个 Givens 变换是指如下形式的一个平面旋转矩阵

$$P_{ij} = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \cos \theta_{ij} & \sin \theta_{ij} & \\ & & & & \ddots & \\ & & & -\sin \theta_{ij} & \cos \theta_{ij} & \\ & & & & & 1 \end{bmatrix}, \quad (3.5.3)$$

其中仅第 i, j 行 (列) 对应的 2×2 子块不是单位阵。容易验证 $P_{ij}P_{ij}^T = I$, 即 P_{ij} 是正交矩阵。从线性变换的观点看, P_{ij} 使 (i, j) 平面作了角度为 θ_{ij} 的旋转。

记 $c_{12} = \cos \theta_{12}$, $s_{12} = \sin \theta_{12}$, 仍用 a_i 表示 A 的第 i 行。考虑

$$A_1 = P_{12}A = \begin{bmatrix} c_{12}a_1 + s_{12}a_2 \\ -s_{12}a_1 + c_{12}a_2 \\ a_3 \\ \vdots \\ a_n \end{bmatrix}. \quad (3.5.4)$$

选取 θ_{12} 使得

$$-a_{11}s_{12} + a_{21}c_{12} = 0. \quad (3.5.5)$$

这样, A_1 在 $(2, 1)$ 位置的元素为零, 前两行元素一般不同于 A , 其余各行与 A 相同。接着可依次计算 $A_2 = P_{13}A_1$, $A_3 = P_{14}A_2, \dots$, 一个接一个地零化第 1 列主对角线以下的元素; 然后再按 $(3, 2), (4, 2), \dots, (n, 2)$ 的顺序零化第 2 列中对应的元素, 如此等等。共计使用

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

个平面旋转矩阵, 最后得到

$$PA \equiv P_{n-1,n} \cdots P_{13}P_{12}A = R, \quad (3.5.6)$$

其中 R 是上三角矩阵, 所有 P_{ij} 均为正交矩阵, 从而 P 也为正交矩阵。因此

$$Q = P^T = P^{-1}, \quad A = QR.$$

注意, 实际计算时无须求出旋转角度 θ_{ij} , 而只须确定 $c_{ij} = \cos \theta_{ij}$ 与 $s_{ij} = \sin \theta_{ij}$, 即可达到所希望的元素零化。例如, 对于式 (3.5.5), 利用隐含条件 $s_{ij}^2 + c_{ij}^2 = 1$, 即得

$$c_{12} = a_{11}(a_{11}^2 + a_{21}^2)^{-1/2}, \quad s_{12} = a_{21}(a_{11}^2 + a_{21}^2)^{-1/2}. \quad (3.5.7)$$

Givens 约化在每级行的更新中具有内在的并行性。在第 1 级, 利用标量运算算出 c_{12} 与 s_{12} 后, 按式 (3.5.4) 更新前两行, 每行需作一个向量数乘运算, 继之一个链三元组运算。因为新的 $(2, 1)$ 位置元素为零, 而用 Givens 变换乘以任一矩阵将不改变各列元素的平方和, 这表明新的 $(1, 1)$ 位置元素为 $(a_{11}^2 + a_{21}^2)^{1/2}$, 故前两行更新运算的向量长度为 $n-1$ 。于是, 约化第 1 列要做 $4(n-1)$ 次向量运算, 向量长度均为 $n-1$ 。类似地, 约化第 j 列要做 $4(n-j)$ 次向量运算, 向量长度均为

$n - j$ 。这样,全部约化平均向量长度为

$$\frac{4(n-1)(n-1) + 4(n-2)(n-2) + \cdots + 4}{4(n-1) + 4(n-2) + \cdots + 4} = O\left(\frac{2}{3}n\right). \quad (3.5.8)$$

与 LU 分解及下段的 Householder 约化相同。

7.4.6 Givens 约化法的向量与并行实现

我们给出一个 Givens 约化算法。

算法 3.14 行向 Givens 约化法

```

for  $i = 1$  to  $n$  do
  for  $j = i + 1$  to  $n$  do
     $c_{ij} \leftarrow a_{ii}(a_{ii}^2 + a_{ji}^2)^{-1/2}$ 
     $s_{ij} \leftarrow a_{ji}(a_{ii}^2 + a_{ji}^2)^{-1/2}$ 
     $\hat{a}_i \leftarrow c_{ij}a_i + s_{ij}a_j$ 
     $\hat{a}_j \leftarrow -s_{ij}a_i + c_{ij}a_j$ 
     $a_i \leftarrow \hat{a}_i, \quad a_j \leftarrow \hat{a}_j$ 
  end for
end for

```

算法 3.14 是按行向处理的 Givens 约化,当然也可写出按列向处理的算法。显然,列向算法的向量化程度不及行向算法 3.14 高。

在并行机上,假定 A 按列环绕交叉存储。我们把约化的顺序称为零化模式。Givens 约化通常的零化模式是零化 A 下三角部分的元素,一次产生一个零,且其在以后各步保持为零,即按照

$$(2, 1), (3, 1), \dots, (n, 1); (3, 2), (4, 2), \dots, (n, 2); \dots; (n, n-1)$$

的次序进行。这里 (p_k, q_k) 指明 Givens 旋转 P_{p_k, q_k} 作用于 $P_{p_{k-1}, q_{k-1}} \cdots P_{p_1, q_1} A$ 以消去其 (p_k, q_k) 元素。

我们希望构造能同时零化多个元素的零化模式。一种此类顺序如下所示: 在第 1 级, 第 1 与第 2 行联立, 同时第 3 与第 4 行联立, 第 5 与第 6 行联立等等; 在第 2 级, 第 1 与第 3 行联立, 同时第 5 与第 7 行联立等等。这样, 每级零化第 1 列中一半余下的非零元素。

Gentleman 提出的标准零化模式将集合

$$S = \{(p, q) \mid 1 \leq q < p \leq n\}$$

中的数对 (p_k, q_k) , $k = 1, 2, \dots, n(n-1)/2$ 按以下次序分解成子集 S_r , $(r = 1, 2, \dots, 2n-3)$:

$$\begin{aligned} &\{(n, 1)\}; \{(n-1, 1)\}; \{(n, 2), (n-2, 1)\}; \{(n-1, 2), (n-3, 1)\}; \\ &\{(n, 3), (n-2, 2), (n-4, 1)\}; \dots \end{aligned}$$

使得

$$S_r = \{(p, q) \mid 1 \leq q < p \leq n, n+2q = p+r+1\}.$$

子集 S_r 中的各个旋转彼此不相交, 可以并行执行, 按此方式既不会引起操作数相关, 也不会破坏先前已消去的零元素。

以 $n = 8$ 为例, Givens 变换操作的并行处理如图所示。其中数字“7”表示在第 7 步可同时消去“7”所在位置的 $a_{21}, a_{42}, a_{63}, a_{84}$ 四个元素, 共需 13 次并行变换。一般地, 对 n 阶矩阵, 当 n 为偶数时共有 $2n - 3$ 级, 最大的并行度出现在第 $n - 1$ 级, 可并行产生 $n/2$ 个零; 当 n 为奇数时也有 $2n - 3$ 级, 最大的并行度出现在第 $n - 1$ 级与第 n 级, 可并行产生 $(n - 1)/2$ 个零。设有足够多的处理机可供使用, 则以上并行计算可用 $n(3n - 2)/2$ 台处理机, 在 $(12n - 18)$ 时间步 (其中含 $2n - 3$ 个平方根运算) 内完成。与串行算法相比, 算术运算部分加速为 $O(n^2/6)$, 平方根运算部分加速为 $O(n/4)$, 平均并行度为 $O(n^2/4)$ 。

然而单个 Givens 变换为典型的向量运算, 不难在实际的并行向量处理机系统上将以上算法多任务化。使用按列环绕交叉存储而且依上述通常约化顺序时, 情况并不理想: 第 1 步只有 P_1 与 P_2 工作, 第 2 步只有 P_1 与 P_3 工作, 等等。这个问题可以通过利用 Givens 约化方法内在的其它类型的并行性加以缓减。

Modi 提出的贪婪算法对集合 S 作另一种分解, 仍以 $n = 8$ 的情形为例。第 1 步可在第 8, 4 行; 第 7, 3 行; 第 6, 2 行和第 5, 1 行之间实行旋转变换, 以同时消去以下位置的 4 个元素 $(8, 1), (7, 1), (6, 1), (5, 1)$ 。显然上述 4 个旋转变换是彼此不相交的。第 2 步可在第 4, 2 行; 第 3, 1 行之间实行旋转变换, 以消去元素 $(4, 1), (3, 1)$; 与此同时, 还可在第 8, 6 行; 第 7, 5 行之间实行旋转变换, 以消去元素 $(8, 2), (7, 2)$ 。这 4 个旋转变换不仅互不相交, 而且它们不破坏前面已产生的零元素。上述操作过程中, 一旦在某确定位置 (p, q) 产生了一个零, 则它再也不会被破坏, 其原理是任何后续的涉及第 p 行的旋转变换均作用于该行与另一行, 设为 p' , 使在位置 (p', q) 上的元素已经是零。第 3 步可同时消去以下元素

$$(2, 1), (6, 2), (5, 2), (8, 3).$$

全过程如图所示。与标准算法相比, 贪婪算法由 13 步降为 11 步完成。两种算法的并行度最大约为 $\lfloor n/2 \rfloor$, 但显然贪婪算法的平均并行度要高, 从而计算时间短。对于长方形矩阵 $A_{m \times n}$ 的 QR 分解, 当 $m \gg n$ 时, 该算法将显示出极大的性能改进。Givens 约化的这些并行计算方案已被证明是数值稳定的, Cosnard 和 Robert 还证明了贪婪算法是最优的并行 Givens 约化方案。

7.4.7 Householder 约化法

Householder 约化法是利用 Householder 变换来实现 QR 分解的。一个 Householder 变换是指如下形式的一个矩阵

$$P = I - ww^T, \quad w^T w = 2. \quad (3.5.9)$$

显然 $P^T = P$, 并且

$$P^T P = I - 2ww^T + w(w^T w)w^T = I.$$

因此, Householder 变换是对称的和正交的, 我们利用此矩阵按下述方式对 A 进行 QR 分解。

首先, 用 A 的第 1 列

$$a^1 = (a_{11}, a_{21}, \dots, a_{n1})^T$$

定义

$$u = (a_{11} - s, a_{21}, \dots, a_{n1})^T, \quad w = \mu u, \quad (3.5.10)$$

其中

$$s = \pm [(a^1)^T a^1]^{1/2}, \quad \gamma = (s^2 - a_{11}s)^{-1}, \quad \mu = \gamma^{1/2}. \quad (3.5.11)$$

并且为了数值稳定, 需选 s 的符号与 a_{11} 的符号相反。于是

$$w^T a^1 = \mu \left[(a_{11} - s)a_{11} + \sum_{i=2}^n a_{i1}^2 \right] = \mu(s^2 - a_{11}s) = \frac{1}{\mu},$$

从而有

$$a_{11} - w_1 w^T a^1 = a_{11} - \frac{\mu(a_{11} - s)}{\mu} = s,$$

$$a_{i1} - w_i w^T a^1 = a_{i1} - \frac{a_{i1}\mu}{\mu} = 0, \quad i = 2, \dots, n.$$

这表明 $P_1 = I - ww^T$ 是 Householder 变换。Householder 变换 P_1 作用于 A 的效果是产生一个新矩阵 A_1 , 它的第 1 列主对角线以下为零, 即

$$A_1 = P_1 A = \begin{bmatrix} s & a'_{12} & \cdots & a'_{1n} \\ 0 & a'_{22} & \cdots & a'_{2n} \\ \vdots & \vdots & & \vdots \\ 0 & a'_{n2} & \cdots & a'_{nn} \end{bmatrix}. \quad (3.5.12)$$

其中对 $j = 2, \dots, n$ 有

$$a'_{ij} = \begin{cases} a_{1j} - \mu(a_{11} - s)w^T a^j, & i = 1, \\ a_{ij} - \mu a_{i1} w^T a^j, & i = 2, \dots, n. \end{cases} \quad (3.5.13)$$

注意: 只要 A 非奇异, 便有

$$a^1 \neq 0, \quad |s| = \left[\sum_{i=1}^n a_{i1}^2 \right]^{1/2} \geq \max_{1 \leq i \leq n} |a_{i1}| > 0.$$

因此, 约化过程是数值稳定的。现在我们在 A_1 中删去第 1 行与第 1 列后的 $(n-1) \times (n-1)$ 子矩阵中重复上述关于 A 的步骤。具体做法是仿照 (3.5.10) 与 (3.5.11) 取

$$u_2 = (0, a_{22} - s_2, a_{32}, \dots, a_{n2})^T, \quad w_2 = \mu_2 u_2,$$

其中

$$s_2 = -\text{sign}(a_{22}) \left[\sum_{i=2}^n (a'_{i2})^2 \right]^{1/2}, \quad \mu_2 = \gamma_2^{1/2}, \quad \gamma_2 = (s_2^2 - a_{22}s_2)^{-1}.$$

定义 $P_2 = I - w_2 w_2^T$, 则 $A_2 = P_2 A_1$ 仍保留 A_1 第 1 列中的零元素, 并且第 2 列的后 $n-2$ 个元素也为零。如此, 继续用 Householder 变换 $P_i = I - w_i w_i^T$, ($i = 3, 4, \dots, n-1$) 逐级将主对角线以下的元素约化为零。最后有

$$P_{n-1} \cdots P_2 P_1 A = R, \quad (3.5.14)$$

其中 R 是上三角矩阵。因所有 P_i 都是正交矩阵, 故

$$P = P_{n-1} \cdots P_2 P_1$$

及 P^{-1} 也是正交的, 并且 $Q = P^{-1}$ 就是 QR 分解中的正交矩阵。(3.5.14) 的自然执行顺序是

$$\begin{cases} A_0 = A, \\ A_k = P_k A_{k-1}, \quad k = 1, 2, \dots, n-1. \end{cases} \quad (3.5.15)$$

7.4.8 Householder 约化法的向量与并行实现

我们可以将上段的论述概括成向量算法:

算法 3.16 列向 Householder 约化法 (内积)

```

for  $k = 1$  to  $n - 1$  do
     $s_k \leftarrow -\text{sign}(a_{kk}) \left( \sum_{i=k}^n a_{ik}^2 \right)^{1/2}$ 
     $\gamma_k \leftarrow (s_k^2 - a_{kk}s_k)^{-1}$ 
     $u_k \leftarrow (0, \dots, 0, a_{kk} - s_k, a_{k+1,k}, \dots, a_{nk})^T$ 
     $a_{kk} \leftarrow s_k$ 
    for  $j = k + 1$  to  $n$  do
         $\alpha_j \leftarrow \gamma_k u_k^T a^j$ 
         $a^j \leftarrow a^j - \alpha_j u_k$ 
    end for
end for

```

在第 k 级, 计算 s_k 要做一个内积, 向量长度为 $n - k + 1$, 对 γ_k 与 $a_{kk} - s_k$ 需做标量运算。内循环是一个内积继之一个链三元组运算, 向量长度也为 $n - k + 1$ 。因此平均向量长度和 LU 分解一样同为 $O(2n/3)$, 而且内循环链三元组的形式也和 LU 分解相同。显然, 与 LU 分解的主要差别是此处内循环还要做内积 $u_k^T a^j$ 。

下面考虑 Householder 约化的另一种方式。记 A_k 的列为 $a_k^1, \dots, a_k^n$, 我们有

$$A_k = (I - w_k w_k^T) A_{k-1} = A_{k-1} - w_k (w_k^T a_k^1, \dots, w_k^T a_k^n). \quad (3.5.16)$$

于是对 $j = k + 1, \dots, n$, A_k 的第 j 列 a_k^j 为

$$a_k^j = a_{k-1}^j - w_k^T a_{k-1}^j w_k = a_{k-1}^j - \gamma_k \mu_k a_{k-1}^j u_k. \quad (3.5.17)$$

从 A_{k-1} 到 A_k , 需要计算的只是 A_{k-1} 中右下 $(n - k + 1) \times (n - k + 1)$ 子矩阵中的元素。

若用 $a_{k,1}, \dots, a_{k,n}$ 表示 A_k 的行, 则式 (3.5.16) 也可改写成

$$\begin{cases} A_k = A_{k-1} - w_k z_k^T, \\ z_k^T = w_k^T A_{k-1} = \mu_k \sum_{i=k}^n u_{k,i} a_{k-1,i} \triangleq \mu_k v_k^T. \end{cases} \quad (3.5.18)$$

于是 A_k 第 i 行

$$a_{k,i} = a_{k-1,i} - w_{k,i} z_k^T = a_{k-1,i} - \gamma_k u_{k,i} v_k^T. \quad (3.5.19)$$

因 $w_k z_k^T$ 是秩为 1 的矩阵, 故有时将 (3.5.19) 称为秩 1 更新方式, 更新可完全由链三元组和线性组合运算完成。由此引出一个向量算法:

算法 3.17 行向 Householder 约化法 (秩 1 更新)

```

for  $k = 1$  to  $n - 1$  do
     $s_k \leftarrow -\text{sign}(a_{kk}) \left( \sum_{i=k}^n a_{ik}^2 \right)^{1/2}$ 
     $\gamma_k \leftarrow (s_k^2 - a_{kk}s_k)^{-1}$ 
     $u_k \leftarrow (0, \dots, 0, a_{kk} - s_k, a_{k+1,k}, \dots, a_{nk})^T$ 

```

```


$$v_k^T \leftarrow \sum_{i=k}^n u_{ik} a_i$$


$$v_k^T \leftarrow \gamma_k v_k^T$$

for  $i = k$  to  $n$  do
     $a_i \leftarrow a_i - u_{ik} v_k^T$ 
end for
end for

```

此算法的平均向量长度仍为 $O(2n/3)$, 向量运算则全为链三元组。然而这里 s_k 的计算不太理想, 这是因为若 A 按行存储, A 的列元素便不在相连的单元之中。对于向量寄存器从存储器输入时, 在第 1 级, 算出的 v_1 可在向量寄存器中保留, 以便所有的 a_i 利用。因此要求输入长为 n 的向量的个数是 $O(2n)$, 基本上与算法 3.16 中采用较好对策的情形相同。

秩 1 更新方式的算法 3.17 是建立在 A 按行存储的基础之上的。在要求向量分量连续存储的向量机上当 A 按列存储时, 可考虑从右边使用 Householder 变换

$$A(I - ww^T). \quad (3.5.20)$$

这样, 第 1 次变换使最后一列主对角线以上元素为零, 相继的变换使第 $n-1$ 列主对角线以上约化, 如此等等。因此, 代之以 A 的 QR 分解是如下形式的 LQ 分解

$$AP_1P_2 \cdots P_{n-1} = L. \quad (3.5.21)$$

其中 L 是下三角矩阵, $Q = (P_1P_2 \cdots P_{n-1})^{-1}$ 为正交矩阵。(3.5.21) 等价于 A^T 的 QR 分解, 不难建立与 QR 分解相应的 LQ 分解算法。

现在, 我们转向讨论 Householder 约化法在并行系统上的实现。假设机器具有局部存储器, 且 A 按列交错存储。先考虑算法 3.16 给出的 Householder 约化法的内积方式, 我们描述一下它的第 1 级 ($k=1$) 的并行步骤:

1. 传送 A 的第 1 列至所有处理机;
2. 处理机 P_1 计算 s_1 , 其它处理机开始计算 $u_1^T a^j$, 传送 s_1 至所有处理机;
3. 所有处理机计算 γ_1 与 $a_{11} - s_1$, 继续 $u_1^T a^j$ 的计算, P_1 中用 s_1 替换 a_{11} ;
4. 所有处理机计算 $\alpha_j = \gamma_1 u_1^T a^j$;
5. 所有处理机计算 $a^j - \alpha_j u_1$ 。

以上步骤的编排可获得好的负载平衡。例如, 当 P_1 计算 s_1 时, 其它处理机便进行各内积 $u_1^T a^j$ 的计算, 尽管它们只有等到能利用 s_1 时才可完成。又如, 采用在所有处理机中计算 γ_1 与 $a_{11} - s_1$, 而不是一台处理机中计算它们然后加以传送。这样, 在此级的通信只是将第 1 列与 s_1 广播到所有处理机。这是第 1 级的主要延迟, 而第 2 级 ($k=2$) 的第 2 列的传送, 一旦第 2 列完成第 1 级的更新即可进行。因此, 并行内积方式约化的每级的基本工作是各列的更新, 而且因其为列交错存储, 直至最后都具有很好的负载平衡。

再考虑算法 3.17 给出的 Householder 约化的秩 1 更新方式。下面是它的第 1 级 ($k=1$) 的并行步骤:

1. 传送 A 的第 1 列至所有处理机;
2. 处理机 P_1 计算 s_1 , 其它处理机开始计算 v_1 , 传送 s_1 至所有处理机;
3. 所有处理机计算 γ_1 与 $a_{11} - s_1$, 继续 v_1 的计算, P_1 中用 s_1 替换 a_{11} ;
4. 所有处理机计算 v_1 ;

5. 所有处理机计算 $a_i - u_{ik}v_1^T$ 。

以上步骤的编排可获得好的负载平衡。例如, 当 P_1 计算 s_1 时, 其它处理机便进行各内积的计算。这样, 在此级的通信只是将第 1 列与 s_1 广播到所有处理机。

